AI INFRA INTERVIEW HANDBOOK / 02

44 Q

大模型训练八股

从反向传播到分布式训练，再到故障恢复

优化器与精度 · 显存账本 · 并行通信

Attention / MoE 算子 · 稳定性 · 性能诊断

> 公开面经筛题，原始论文与官方文档校验
>
> 每题包含：口述答案 / 背景 / 原理 / 实现 / 追问
>
> 附 CPU 参考代码、数值检查与可点击来源

检索与整理截至 2026-10-31

面向大模型算法、训练框架与 RL Infra 面试准备

独立技术整理 · 不代表任何公司的官方题库

阅读说明 / SCOPE & EVIDENCE

大模型训练八股

这不是仅有题目和关键词的清单。每题按“能口述 → 能解释 → 能实现 → 能验证”的顺序组织，服务于大模型算法与 AI Infra 岗位中的连续追问。

检索范围与边界

本次跨平台检索并筛选了 17 条公开面经、转载与社区题库，主要可读材料来自牛客、GitHub、CSDN、博客园、AIInfraGuide 和独立技术站点。本册关联其中 11 条记录，并列出 40 项技术依据。不是对“全网所有面经”的穷尽抓取，也不宣称覆盖登录后、付费或未能公开访问的内容。

先按 RL / 后训练 / 训练框架 / 算子 / 分布式等关键词找候选，再区分作者自述、转载和题库；合并相同主题，最后用原始论文及官方文档写答案。未将转载次数转换为出题频率。

来源标签	如何解读
A · 作者自述	作者声称参加过对应面试。岗位、公司和面试日期仅按原帖记录，不是企业认证。
B · 转载 / 题库	用于发现问题和查漏补缺；不能当作独立面试样本，也不直接采信其中答案。
延伸题	为补全知识链或工程能力加入，未声称该具体题目在已检索面经中出现。

优先级与引用规则

本册 44 题中有 36 题标为 P0，其余为 P1。P0 表示建议先掌握的基础和岗位核心，P1 表示进阶补充；这是学习建议，不是统计学上的高频概率。题号中的“同类主题”只说明公开记录覆盖相近方向，不代表本册题目与原帖逐字相同。

每题底部链接到技术依据；来源目录保留原文链接、日期口径与证据等级。页面访问时间统一为 2026-10-31；只有月日的帖子不推定年份。官方 latest/main 页面可能继续变化，复现时请另锁定软件版本与 commit。

> **验证边界：**正文中的算量、显存与设备配置均写明假设。附录仅进行了 CPU 数学与代码测试，未运行多 GPU 训练、NCCL 性能或真实招聘流程；不提供虚构加速比。

建议读法

第一遍只看每题的“口述版”；第二遍推导公式并完成数值例子；第三遍遮住答案完成追问和代码。项目面试中，用真实日志、trace、对照实验与失败案例替代只背框架名。

目录 / CLICK TO NAVIGATE

问题导航

01 训练数学与模型

01 预训练和 SFT 的目标怎么写？为什么 labels 要 shift？ 6
02 Transformer 的 Attention、GQA、SwiGLU、Norm、RoPE 分别算什么？ 7
03 Autograd 如何构图？leaf、detach、no_grad 和 in-place 有什么区别？ 8
04 手推线性层与 SwiGLU 反向，dgrad 和 wgrad 的 GEMM 形状？ 9
05 SGD、Adam、AdamW、LAMB 有什么不同？优化器状态占多少？ 10
06 有效 batch、梯度累积、warmup 与学习率缩放怎样理解？ 11
07 参数量和训练 FLOPs 如何估算？6PT 什么时候会失真？ 12

02 显存与精度

08 训练显存账本怎么列？16 bytes/参数是通用结论吗？ 13
09 激活显存如何随 batch、序列、hidden 增长？FlashAttention 省在哪里？ 14
10 FP16、BF16、AMP 与 GradScaler，为什么 BF16 通常不需要 loss scaling？ 15
11 FP8、block scaling、MXFP8 的区别，训练为何比推理更难？ 16
12 梯度裁剪怎么做？分片参数的全局范数如何求？ 17

03 分布式与通信

13 DDP 如何同步梯度？bucket、hook、unused parameters 有什么影响？ 18
14 ZeRO 1/2/3 分别切什么？能手算每卡模型状态吗？ 19
15 FSDP 与 ZeRO-3 有什么共同点？FSDP2 要关注哪些边界？ 20
16 CPU/NVMe offload 与 Hybrid Sharding 如何选？ 21
17 TP 的列并行和行并行，前向与反向各通信什么？ 22
18 Attention 的 TP 怎么切？GQA 的 KV heads 少于 TP 时怎么办？ 23
19 PP 的 GPipe、1F1B、microbatch 与 bubble 如何计算？ 24
20 Interleaved pipeline、virtual stage 与 zero-bubble 思路是什么？ 25
21 SP、CP、Ulysses、Ring Attention 分别切什么、传什么？ 26
22 MoE 训练中 EP、dispatch、combine、capacity 与负载均衡怎么解释？ 27
23 AllReduce、ReduceScatter、AllGather、AlltoAll 的区别与通信量？ 28
24 GPU 拓扑、NVLink、PCIe、NUMA、RDMA 对并行布局有什么影响？ 29
25 通信计算 overlap 为什么“开了 async”仍可能没加速？ 30
26 多维并行 world size 如何计算？EP 为什么不能总额外乘进去？ 31

目录 / CLICK TO NAVIGATE

问题导航 · 续

04 算子、内存与编译

27 梯度累积遇到不同长度，怎样保证全局 token mean 正确？ 32
28 Activation checkpoint 的原理、选择性重算与 RNG 风险？ 33
29 FlashAttention 与 Online Softmax 如何推导？LSE 怎么合并？ 34
30 Register、shared memory、occupancy 与 persistent pipeline 怎么权衡？ 35
31 Grouped GEMM 的 contiguous / masked 布局及 tile scheduling？ 36
32 torch.compile、算子融合与 CUDA Graph 分别解决什么？ 37

05 数据、稳定性与恢复

33 数据 pipeline、tokenization、packing、mask 如何影响训练质量和吞吐？ 38
34 LoRA 与 QLoRA 省哪些显存？冻结基座是否可以 no_grad？ 39
35 OOM、显存持续增长与 fragmentation，怎么定位而不是乱减 batch？ 40
36 Loss spike、NaN、梯度爆炸如何做第一现场排查？ 41
37 NCCL timeout/hang 怎么查？为什么不先把 timeout 调大？ 42
38 设随机种子就确定性了吗？如何验证训练优化没有改变结果？ 43
39 分布式 checkpoint 怎么保证一致？异步保存会不会读到更新中的参数？ 44

06 性能测量与系统设计

40 MFU、HFU、GPU utilization、tokens/s 分别说明什么？ 45
41 强扩展、弱扩展和 Amdahl 定律如何解释多卡不加速？ 46
42 手写题：一个不会错位、错误累积或漏同步的训练 step 47
43 系统设计：7B 模型、8×80GiB 预算，该如何选 DDP / ZeRO / TP？ 48
44 系统设计：70B、128 卡、多节点训练如何规划与验收？ 49

附录与来源

参考代码与边界说明 50
实际 CPU 验证结果 53
复习路径与模拟面试 54
公开面经与题库来源 55
论文与官方技术依据 57

检索结果如何变成题目

面经主题与本册映射

下表是主题索引，不是各家公司面试通过率或出题频率。回答内容经过重写和技术核验，没有复制某篇面经的完整答案。

知识模块	公开主题记录	题号
训练数学与模型	[I05]、[I06]、[I07]、[I10]、[I11]、[I12]、[I13]、[I16]	01–07
显存与精度	[I02]、[I05]、[I06]、[I07]、[I13]、[I16]、[I17]	08–12
分布式与通信	[I02]、[I05]、[I06]、[I10]、[I13]、[I16]	13–26
算子、内存与编译	[I05]、[I06]、[I07]、[I12]、[I13]、[I16]	27–32
数据、稳定性与恢复	[I06]、[I07]、[I11]、[I12]、[I13]、[I16]	33–39
性能测量与系统设计	[I05]、[I06]、[I10]、[I12]、[I13]、[I15]、[I16]	40–44

TP 行列切分 → SP → 通信

从矩阵 shape 推导，不只背 AllReduce 次数。 同类主题：[I05] [I10]。

优化器 → 精度 → ZeRO

声明 bytes/parameter 假设，再算状态分片。 同类主题：[I06] [I16]。

FlashAttention → Softmax → CUDA

区分算量、IO、布局与实际硬件资源。 同类主题：[I05] [I07]。

项目 → 吞吐 → 容错

报告关键路径、回归测试和可恢复状态。 同类主题：[I06] [I13]。

> 同一作者的不同面试保留为不同经历，但不当成独立作者样本；多平台搬运只作为一个线索。帖子中的薪资、岗位排名、通过率以及未经证实的“必考”结论未纳入本册。

01 训练数学与模型 / QUESTION 01

01 预训练和 SFT 的目标怎么写？为什么 labels 要 shift?

P0 · 优先掌握 公开记录同类主题：[I11] [I12] [I16]

> **面试口述版**
>
> 因果语言模型用位置 t 的 logits 预测 token t+1。预训练通常对有效文本 token 做交叉熵，SFT 常只对 assistant 回答位置计损失；attention mask 控制可见性，loss mask 控制哪些位置贡献目标，两者不能混为一谈。

背景：问题为什么会出现

张量 logits 为 [B,S,V]，labels 为 [B,S]。常见实现取 logits[:,:-1] 和 labels[:,1:]；若框架内部已经 shift，外部再 shift 就会错一位。输入 token 本身作为下一位置目标，是自回归 teacher forcing 的对齐方式。

原理与推导：把关键条件说清楚

稳定 CE 通过 logsumexp 计算 −z_target+logΣexp(z)。SFT 中 prompt 仍会影响回答，并不因为不计 prompt loss 就从前向移除。多轮对话可只训最后一轮，也可训多轮 assistant span，需明确训练定义；padding 通常用 ignore_index。

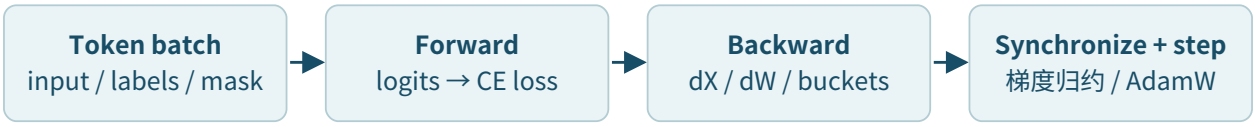
$$L = -\Sigma_{b,t}\, m_{b,t} \log p\theta(x_{b,t+1}|x_{b,\le t}) \,/\, \Sigma_{b,t}\, m_{b,t}$$

实现方案与验证

手算序列 [BOS, A, B, EOS]：BOS 位置预测 A，A 预测 B，B 预测 EOS。构造小 vocab 的高正确 logits，正确 shift 时 loss 应很低，错位时明显升高。验证所有被屏蔽目标位置的 logit 梯度为零，以及全无有效目标时上层如何处理。

常见追问与易错点

追问：padding token 等于 EOS 能直接用 token ID mask 吗？答：可能把真实 EOS 一起屏蔽，应用有效长度/attention mask 和标签语义。追问：SFT loss 降就是任务更好？答：仍需 held-out 和生成评估。



注意：loss mask 与 attention mask 控制不同语义。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T01] [T05] ｜ 口述、数值例子及实验建议为本册组织与推导。

01 训练数学与模型 / QUESTION 02

02 Transformer 的 Attention、GQA、SwiGLU、Norm、RoPE 分别算什么?

P0 · 优先掌握 公开记录同类主题：[I05] [I10] [I11]

> **面试口述版**
>
> Attention 在 token 间混合信息，FFN/SwiGLU 在每个 token 的通道上非线性变换；GQA 共享部分 K/V heads 以减少 KV 规模；Norm 稳定激活尺度，RoPE 通过旋转 Q/K 引入相对位置关系。面试要能说出张量形状，而不只是模块名。

背景：问题为什么会出现

设 hidden=d，query heads=hq，KV heads=hkv，head_dim=dh。Q 投影输出 hq · dh，K/V 各输出 hkv · dh；在常见设计中 hq · dh=d。Attention 是 softmax($QK^T/\sqrt{dh}$+mask)V，再经输出投影。

原理与推导：把关键条件说清楚

SwiGLU 常用 gate=XWg、up=XWu、hidden=SiLU(gate)⊙up、out=hidden · Wd，三组矩阵不是传统两层 FFN 的同一参数账本。RMSNorm 以均方根缩放而不减均值。RoPE 对成对通道旋转，使 Q/K 内积依赖位置差；不意味着无需长上下文训练就可无限外推。

$$\mathrm{Attn}(Q,K,V)=\mathrm{softmax}(QK^T/\sqrt{dh}+\mathrm{mask})V \;;\; \mathrm{SwiGLU}(X)=(\mathrm{SiLU}(XWg)\odot XWu)Wd$$

实现方案与验证

画出 [B,S,d]→[B,hq,S,dh] 的转置与布局；检查 GQA 的 query-to-KV head 映射、causal mask、position IDs。做逐模块 reference 对照，再融合 kernel；模型 forward close 后仍需 backward 验证。

常见追问与易错点

追问：GQA 是否把 QK^T 算量按 hkv/hq 一起缩小？答：query heads 仍各自计算 attention，主要减少 K/V 投影和 cache，不可直接按同一比例算全部 FLOPs。追问：MoE 替换 Attention 吗？答：常见 MoE 替换的是 FFN。

技术依据：[T01] [T02] [T03] [T04] | 口述、数值例子及实验建议为本册组织与推导。

01 训练数学与模型 / QUESTION 03

03 Autograd 如何构图？leaf、detach、no_grad 和 in-place 有什么区别？

P0 · 优先掌握 公开记录同类主题：[I07] [I12]

> **面试口述版**
>
> Autograd 在前向记录可微操作与反向所需张量，backward 按链式法则累计梯度。detach 切断图连接；no_grad 不记录区域内图；冻结参数只是不计算该参数梯度，若需要输入梯度，运算仍可能必须被记录。

背景：问题为什么会出现

叶子参数通常持有 .grad，非叶张量默认不保留 .grad，除非显式 retain_grad。多次 backward 会累加而非自动覆盖。in-place 改写可能破坏反向保存的版本，尤其 view/共享存储更容易出问题。

原理与推导：把关键条件说清楚

例：Y=XW，W 冻结但 X 来自可训练 LoRA 或前层，仍需 $dX=dY \cdot W^T$。因此把整个冻结层放入 no_grad 可能切断需要的梯度。相反，记录 loss.item() 可避免把每一步的计算图留在 Python 列表里；记录 loss 张量本身则可能导致显存持续增长。

实现方案与验证

定位无梯度时逐层检查 requires_grad、grad_fn、参数是否真的在 optimizer 内、是否误 detach 或把 tensor 转成 Python 数再构造回来。定位图泄漏时查看引用、hooks、缓存和保留的输出。对自定义 Function 使用 gradcheck，而不只测 forward。

常见追问与易错点

追问：retain_graph=True 能修复二次 backward 报错吗？答：它保留图并增加内存，但是否需要二次反传要先确认；盲加可能掩盖错误数据流。追问：zero_grad(set_to_none=True) 与填零完全同义吗？答：内存与“无梯度”的优化器行为可能不同。

技术依据：[T05] | 口述、数值例子及实验建议为本册组织与推导。

01 训练数学与模型 / QUESTION 04

04　手推线性层与 SwiGLU 反向，dgrad 和 wgrad 的 GEMM 形状?

P0 · 优先掌握　公开记录同类主题：[I05] [I07]

> **面试口述版**
>
> 对 Y=XW，dX=dY · W^T，dW=X^T · dY，db 是 dY 沿样本维求和。前向、dgrad、wgrad 的 reduction 维不同，所以同一优化参数不一定同时适用于三者；MoE wgrad 的归约维还是各专家接收的 token 数。

背景：问题为什么会出现

设 X=[M,K]，W=[K,N]，Y=[M,N]：前向 MNK，dX 是 [M,N]×[N,K]，dW 是 [K,M]×[M,N]。训练反向大致需要两个同量级 GEMM，构成“前向+反向约三倍前向矩阵乘”的基础直觉。

原理与推导：把关键条件说清楚

SwiGLU 令 G=XWg、U=XWu、H=SiLU(G)⊙U。来自 down 层的 dH 先分成 dU=dH⊙SiLU(G) 与 dG=dH⊙U⊙SiLU′(G)，再分别计算参数梯度和两路 dX 并求和。不能忘掉 gate 的 sigmoid 导数或两路输入梯度相加。

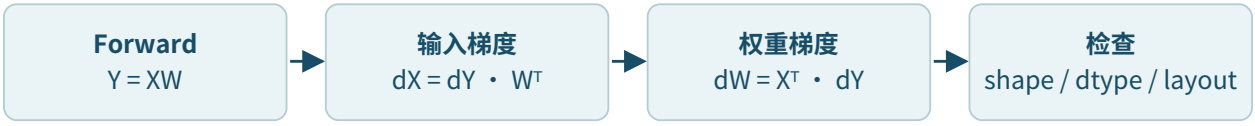
$$dX = dY\,W^T ;\ dW = X^T\,dY ;\ SiLU'(g)=\sigma(g)+g\sigma(g)(1-\sigma(g))$$

实现方案与验证

附录用 float64 对线性层手推梯度与 autograd 比较。进一步用有限差分检查少量参数，步长需匹配精度。GPU 测试把不同 M/N/K、非对齐、零 token expert 和多个累积 microbatch 纳入，验证累加顺序与输出 dtype。

常见追问与易错点

追问：为什么 wgrad 容易受小 batch 影响？答：M 作为归约维较短，kernel 布局和分块效率可能改变。追问：参数没训练还要 wgrad 吗？答：通常不需要，但 dgrad 是否需要由上游梯度路径决定。



这里 W 按 [in,out] 记；PyTorch Linear 通常存储转置权重。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T05] [T03] ｜ 口述、数值例子及实验建议为本册组织与推导。

01 训练数学与模型 / QUESTION 05

05 SGD、Adam、AdamW、LAMB 有什么不同？优化器状态占多少？

P0 · 优先掌握 公开记录同类主题：[I06]

> **面试口述版**
>
> SGD 按梯度更新，可加动量；Adam 用一二阶矩做自适应缩放；AdamW 将 weight decay 与自适应梯度更新解耦；LAMB 在此类更新上加入层级尺度调整以适应大 batch。算法选择与状态 dtype 共同决定显存。

背景：问题为什么会出现

Adam 的 m、v 分别是梯度与平方梯度的指数滑动平均，并有偏差校正。若两者 FP32，每个训练参数仅这两份状态就占 8 bytes。低精度参数是否配 FP32 master copy 是实现决策，不是“Adam 必须四份同精度张量”。

原理与推导：把关键条件说清楚

把 L2 项 λθ 加进梯度再经过 Adam 的自适应缩放，不等价于每步直接做 θ←(1−ηλ)θ 的 decoupled decay。层级 trust ratio 也不等价于“学习率直接变大”。讨论优化器时应说明 eps、betas、decay 排除规则与 state 初始化。

$$m_t=\beta1 m_{t-1}+(1-\beta1)g_t\ ;\ v_t=\beta2 v_{t-1}+(1-\beta2)g_t^2$$

实现方案与验证

检查第一次 optimizer.step 是否惰性创建状态并触发显存峰值；使用 fused/foreach 时还要计临时 buffer。做单参数标量参考验证数值更新，检查梯度为 None 与零梯度时 decay 行为。跨 checkpoint 恢复要带 optimizer state，否则并非连续训练。

常见追问与易错点

追问：AdamW 一定比 Adam 更好吗？答：不能脱离任务、配方和超参。追问：混合精度把参数改 BF16 就能把优化器显存减半吗？答：状态可能仍 FP32，必须查看实际 dtype。

技术依据：[T06] [T35] [T36] ｜ 口述、数值例子及实验建议为本册组织与推导。

01 训练数学与模型 / QUESTION 06

06 有效 batch、梯度累积、warmup 与学习率缩放怎样理解?

P0 · 优先掌握 公开记录同类主题：[I06] [I11]

> **面试口述版**
>
> 固定每卡样本数时，有效 batch≈micro_batch×accumulation×DP；TP/PP 不提供新的独立样本，不能都乘进去。warmup 让早期不稳定的优化状态逐渐进入目标学习率，但具体曲线与放大 batch 后的学习率规则都需实验验证。

背景：问题为什么会出现

当序列长短不一，样本数相同不代表有效 token 数相同。调参应报告训练 token budget 和每次 optimizer update 的有效目标 token。scheduler 以 optimizer step 还是 token 数推进，需要与累积设置一致。

原理与推导：把关键条件说清楚

例：DP=8、每卡 microbatch=2、累积16次，global batch=256 条；若平均有效长度从1K变2K，同样一步训练 token 数翻倍。把 accumulation 从16改8同时未调 scheduler，按样本/按token看的训练阶段都会变化。

```
B_global = B_micro × N_accum × D_DP （相同长度/样本口径下）
```

实现方案与验证

等权固定 microbatch 时可 loss/acc_steps；不等长且追求全局 token mean 时应使用窗口总有效计数。避免每个 microstep 都执行 optimizer.step 或 scheduler.step。DDP no_sync 可减少中间同步，但必须保留最后一次正确归约并满足框架语义。

常见追问与易错点

追问：梯度累积严格等价大 batch 吗？答：仅在目标归一化一致、无跨样本状态等假设下近似或相同；dropout、batchnorm、浮点顺序和梯度裁剪时机会影响。追问：warmup 能解决坏数据 NaN 吗？答：不能。

技术依据：[T06] [T09] [T12] | 口述、数值例子及实验建议为本册组织与推导。

01 训练数学与模型 / QUESTION 07

07　参数量和训练 FLOPs 如何估算？6PT 什么时候会失真?

P0 · 优先掌握　公开记录同类主题：[I05] [I10] [I13]

> **面试口述版**
>
> 稠密模型常用训练 FLOPs≈6PT 作一阶估计：参数相关矩阵乘前向约2PT，反向约4PT。该式需要说明参数定义与 FLOP 计数 convention，长序列 attention、重计算、稀疏 MoE、词表投影等都可能让简单估计失真。

背景：问题为什么会出现

这里 P 是参与计算的参数规模，T 是训练有效 token 总数，乘加按2 FLOPs计。MHA 的 QK^T和AV 有序列平方项，不由固定 P 完全表达；embedding lookup 也不等于把 embedding 参数全做一次 GEMM。

原理与推导：把关键条件说清楚

原创算例：7×10^9 参数、10^9 token，6PT≈4.2×10^19 FLOPs，不包括重计算等附加成本。SwiGLU 的三矩阵参数约3df；MHA 权重近似4d^2（GQA另算）；MoE需区分总专家参数与每token激活专家参数，不能拿总P直接套稀疏计算。

```
C_train ≈ 6PT ; GEMM(M,K,N) ≈ 2MKN FLOPs
```

实现方案与验证

估算先列模块形状与前/反向 GEMM，再与 profiler 或框架 FLOPs 估计对齐。报告 useful/model FLOPs 和实际执行 FLOPs 的区别；padding、重算与不必要 kernel 是真实耗时，但不能全算成有用训练进展。

常见追问与易错点

追问：减少参数一定更快吗？答：算术强度、访存、通信和batch形状可能主导。追问：6PT能直接算训练时间吗？答：还需要持续有效吞吐，并考虑数据、通信、checkpoint和故障等待。

技术依据：[T37] [T01] [T03]　|　口述、数值例子及实验建议为本册组织与推导。

02 显存与精度 / QUESTION 08

08 训练显存账本怎么列？16 bytes/参数是通用结论吗？

P0 · 优先掌握 公开记录同类主题：[I02] [I06] [I16]

> **面试口述版**
>
> 不是通用结论。先列参数、梯度、master 参数、优化器状态，再加激活、通信 buffer、临时工作区和碎片。16 bytes/参数只对应一组常见混合精度 Adam 假设，BF16 实现、梯度 dtype 和状态分片都会改变它。

背景：问题为什么会出现

示例账本：参数BF16=2、梯度BF16=2、master FP32=4、m/v FP32=8，共16 bytes。若参数FP32且无独立master，参数4+梯度4+m/v8也为16；若BF16参数/梯度且无master则为12。相同总量不代表内部结构相同。

原理与推导：把关键条件说清楚

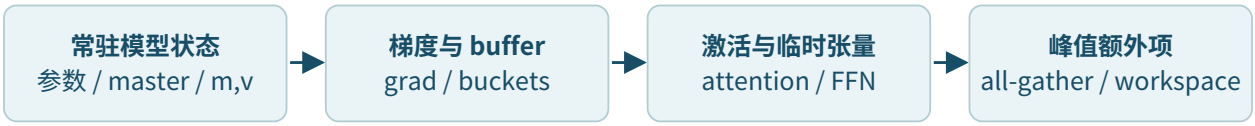
7B×16 bytes=112 GB≈104.3 GiB；7B×12=84 GB≈78.2 GiB。此处B表示10^9参数，GiB为2^30bytes。模型“能装进某卡”的判断必须再加峰值激活、通信聚合、optimizer临时副本和框架开销。

实现方案与验证

列常驻/峰值两栏并区分 allocated 与 reserved。分别测模型加载、第一轮前向、反向、首次step和checkpoint阶段。需要检查 optimizer惰性初始化与FSDP层级all-gather峰值，不能用训练稳定后的显存截图代表全生命周期。

常见追问与易错点

追问：为什么省了参数显存仍OOM？答：长序列激活、logits、KV或临时gather可能主导。追问：各项都除以GPU数行吗？答：只有相应被分片的状态能除，复制项和通信峰值不能平均幻想。



先算账，再对照 memory snapshot；allocated 与 reserved 不是同一个指标。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T08] [T32] [T07] ｜ 口述、数值例子及实验建议为本册组织与推导。

02 显存与精度 / QUESTION 09

09 激活显存如何随 batch、序列、hidden 增长？FlashAttention 省在哪里？

P0 · 优先掌握 公开记录同类主题：[I05] [I06]

> **面试口述版**
>
> 激活取决于层数、microbatch、序列长度、hidden与具体保存策略。传统 attention 若保存分数/概率可有 $B \cdot h \cdot S^2$ 项；FlashAttention 避免完整分数矩阵落 HBM，但精确稠密 attention 的算量仍通常随 S^2 增长。

背景：问题为什么会出现

训练需要为 backward 保存输入或中间值；推理只有前向不等于相同激活账本。FFN 扩展维、QKV、norm与dropout状态、词表logits都可能成为主要项。很多“激活公式”暗含某个实现，不能脱离重计算与融合策略套常数。

原理与推导：把关键条件说清楚

教学对照：序列长度翻倍、其余不变，线性激活项约翻倍，显式attention矩阵约变4倍。即使用FlashAttention去掉主要S^2存储，attention计算仍可能约4倍；而增加CP卡数分摊序列会引入通信，并非把整个模型状态都按序列卡数均摊。

$$M_{saved} \approx \text{linear terms in } B \cdot S \cdot d \cdot L + \text{optional attention terms in } B \cdot h \cdot S^2 \cdot L$$

实现方案与验证

用峰值memory snapshot和逐模块测量验证哪个张量活得最久，再选择activation checkpoint、selective recompute、sequence/context parallel或logits分块。对长context先减少microbatch而保持全局目标，再比较总wall time与重算开销。

常见追问与易错点

追问：activation checkpoint是否把激活归零？答：仍保留边界输入、必要状态和重计算临时张量。追问：FlashAttention是近似attention吗？答：其标准形式计算同一数学attention，但浮点顺序不同不保证bitwise一致。

技术依据：[T18] [T20] [T01] | 口述、数值例子及实验建议为本册组织与推导。

02 显存与精度 / QUESTION 10

10 FP16、BF16、AMP 与 GradScaler，为什么 BF16 通常不需要 loss scaling?

P0 · 优先掌握 公开记录同类主题：[I06] [I17]

> **面试口述版**
>
> FP16与BF16都16位，但BF16指数位更多、范围接近FP32，FP16精度位更多却更容易小梯度下溢或大值溢出。AMP按算子选择计算精度，GradScaler缩放loss保护FP16小梯度；它不是让所有张量统一半精度。

背景：问题为什么会出现

前向矩阵乘可以低精度，归约、softmax、norm和优化状态常保留更高精度。BF16通常不需要loss scaling的理由是动态范围，并不意味着BF16不会NaN或不会有舍入误差。

原理与推导：把关键条件说清楚

缩放loss为sL会得到sg，step前需unscale。梯度裁剪必须在unscale后，否则阈值被无意乘/除s。发现inf/NaN时scaler可能跳过step；scheduler若照常前进，则学习率进度与真正参数更新次数发生偏离。

实现方案与验证

比较FP32小模型reference，检查每层激活/梯度range，再逐步启用autocast。监控scaler scale、跳步次数、grad norm和loss。梯度累积时一个有效batch内scale必须一致；不要每个microbatch自行unscale/step打破语义。

常见追问与易错点

追问：参数全转half就是AMP吗？答：不是，会改变master/state与敏感算子的行为。追问：BF16在FP16硬件路径上能随便替换吗？答：须核对硬件和kernel支持，不能只看存储字节相同。

技术依据：[T07] [T19] | 口述、数值例子及实验建议为本册组织与推导。

02 显存与精度 / QUESTION 11

11 FP8、block scaling、MXFP8 的区别，训练为何比推理更难?

P1 · 进阶补充 公开记录同类主题：[I17] [I07]

> **面试口述版**
>
> FP8描述数据格式，block scaling描述共享scale的粒度；MXFP8是有特定32值分组及E8M0尺度约定的微缩放方案。它本身就是block scaling的一种，不能把“MXFP8”和“所有分块FP8”当成互斥类别。

背景：问题为什么会出现

训练不仅量化forward激活/权重，还涉及梯度、转置布局、wgrad、累计状态和收敛。scale计算与转换也是开销，矩阵乘峰值提高不保证整步等比例加速。E4M3/E5M2的范围与精度取舍也应结合张量角色说明。

原理与推导：把关键条件说清楚

MXFP8每32个数据配一个8位scale，理想只算数据+scale存储为32+1 bytes，比32个BF16的64 bytes小，但不是正好一半；忽略padding与布局。转置后分组轴改变，通常不能把量化后的数据简单转置就当同一量化方案，需从较高精度源构建对应布局。

实现方案与验证

验证正反向误差分布、异常值饱和、scale更新范围、long-run loss和held-out质量。计时包含cast/scale/transpose与通信格式转换，并按真实shape分布比较。只用一个大方阵GEMM数字回答“训练加速多少”不足以成立。

常见追问与易错点

追问：scale能与MMA重叠吗？答：部分准备可流水线，但受数据依赖、TMEM/SMEM和硬件指令接口约束。追问：换FP8后OOM一定缓解一半？答：优化器、master与部分激活可能仍高精度。

技术依据：[T19] [T29] | 口述、数值例子及实验建议为本册组织与推导。

02 显存与精度 / QUESTION 12

12 梯度裁剪怎么做？分片参数的全局范数如何求？

P0 · 优先掌握 公开记录同类主题：[I06] [I13]

> **面试口述版**
>
> 全局范数裁剪把所有参数梯度按同一系数缩放，以限制一次更新的梯度量级。分片状态需要正确聚合各独立分片的平方和；复制参数若重复计数会把范数高估。裁剪位置应在累积完成、同步及必要的unscale之后。

背景：问题为什么会出现

对L2范数，norm=$\sqrt{\Sigma_j \|g_j\|^2}$，系数c=min(1,max_norm/(norm+ε))。这不同于逐元素clip，也不同于每个层独立裁剪。后两者改变方向和相对比例，属于另一种策略。

原理与推导：把关键条件说清楚

示例两个独立shard梯度分别[3]与[4]，全局norm=5，阈值1则均乘0.2。若每rank只用本地norm，会得到[1]与[1]，不再是同一个全局裁剪结果。若两rank是同一梯度副本，则不应再次把3^2+4^2式思路用于复制计数。

$$g \leftarrow g \cdot \min(1, \tau / (\|g\|_2+\varepsilon))$$

实现方案与验证

使用支持DTensor/FSDP语义的接口或明确mesh去重规则；先用小模型与未分片参考对比。记录裁剪前norm、裁剪频率和更新/参数范数比；频繁强裁剪应回查数据、loss缩放、学习率与异常梯度，而不只提高阈值。

常见追问与易错点

追问：裁剪能修NaN吗？答：norm已经NaN时不能可靠修复，应阻止错误step并定位首个异常。追问：每个microbatch裁剪等价最后裁剪吗？答：不等，clip是非线性操作。

技术依据：[T07] [T09] [T10] ｜ 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 13

13 DDP 如何同步梯度？bucket、hook、unused parameters 有什么影响？

P0 · 优先掌握 公开记录同类主题：[I05] [I06]

> **面试口述版**
>
> DDP在各rank保存模型副本，前向用不同数据，反向梯度就绪后通过bucket通信并按语义归约。bucket把小梯度打包以减少通信启动成本，也决定通信何时能与backward重叠。DDP本身不切分参数或自动切分输入数据。

背景：问题为什么会出现

若等大小batch上求本地mean loss，默认平均梯度对应全局样本mean。若不同rank有效token数量不同，则需要另行权重校正。optimizer通常各rank用一致梯度更新，不靠每步broadcast完整模型来保证一致。

原理与推导：把关键条件说清楚

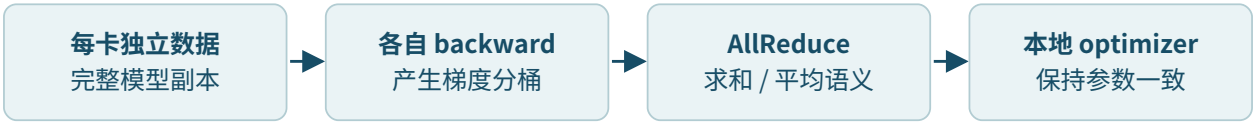
小bucket更早发但启动次数多，大bucket带宽效率较好却可能晚到反向尾部才发。某参数某轮未参与loss时，相关梯度可能不就绪；动态分支和各rank不同控制流必须按框架选项正确处理，不能一概开启或关闭unused检测。

实现方案与验证

观察gradient ready、collective launch与GPU kernel时间线，判断真正暴露的通信尾巴。使用DistributedSampler并正确设置epoch/seed，确认各rank步数相同。梯度累积用no_sync时也要保证forward在正确上下文，并在最后microstep完成同步。

常见追问与易错点

追问：DDP同步loss吗？答：通常同步梯度，日志loss另处理。追问：多卡loss比单卡差一定是通信误差？答：先看有效batch、数据顺序、归一化与学习率，数值误差只是原因之一。



有效 token 数不等时，平均局部 mean loss 会改变全局目标权重。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T09] [T16] ｜ 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 14

14 ZeRO 1/2/3 分别切什么？能手算每卡模型状态吗？

P0 · 优先掌握 公开记录同类主题：[I06] [I16] [I02]

> **面试口述版**
>
> ZeRO-1切优化器状态，ZeRO-2再切梯度，ZeRO-3再切参数；它们减少DP副本冗余，而不是直接把所有激活也平均切开。参数按需聚合和临时buffer会使峰值高于常驻公式。

背景：问题为什么会出现

采用参数2P、梯度2P、master+m/v共12P bytes的示例，D路分片：普通DP为16P；Z1=4P+12P/D；Z2=2P+14P/D；Z3≈16P/D。实际要用真实dtype重列，不能对任意BF16训练都套16P。

原理与推导：把关键条件说清楚

原创算例P=7B、D=8：DP112GB，Z1 38.5GB，Z2 26.25GB，Z3常驻14GB；均为十进制，不含激活和临时gather。由此可见“Z1固定省4/3、Z2固定省2倍”并不普遍成立，节省比例依赖D与账本。

```
M_DP=16P ; M_Z1=4P+12P/D ; M_Z2=2P+14P/D ; M_Z3≈16P/D
```

实现方案与验证

当Z2已满足显存且通信成为关键路径时，Z3未必更快；当参数复制限制容量时，Z3才可能是必要选项。看每层all-gather峰值、prefetch、持久参数和bucket大小；用同一global batch与checkpoint策略比较端到端step。

常见追问与易错点

追问：ZeRO-3可训练任意大层吗？答：完整计算单元仍需聚合或采用其他切分，最大单元和临时工作区约束仍在。追问：offload属于哪一阶段？答：它是状态存放层级的选择，可与不同分片策略组合。

方案	每卡状态字节（示意）	含义
DDP	16P	参数、梯度、优化器均复制
ZeRO-1	4P + 12P/D	仅优化状态分片
ZeRO-2	2P + 14P/D	再分片梯度
ZeRO-3	16P/D + 临时峰值	再分片参数；不含激活

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T08] [T32] ｜ 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 15

15 FSDP 与 ZeRO-3 有什么共同点？FSDP2 要关注哪些边界？

P0 · 优先掌握 公开记录同类主题：[I06] [I13]

> **面试口述版**
>
> 两者都有分片模型状态、计算前聚合参数的思路，但API、调度、参数表示、通信与生态集成不同。FSDP2采用基于DTensor的per-parameter分片等设计；回答时区分FSDP1与2，不把旧FlattenParameter细节套到所有版本。

背景：问题为什么会出现

典型full shard路径：前向前all-gather当前单元参数；前向后按配置reshard；反向前可能再次gather；梯度计算后reduce-scatter；优化器更新本地shard。不同reshard/prefetch设置改变常驻显存与通信量。

原理与推导：把关键条件说清楚

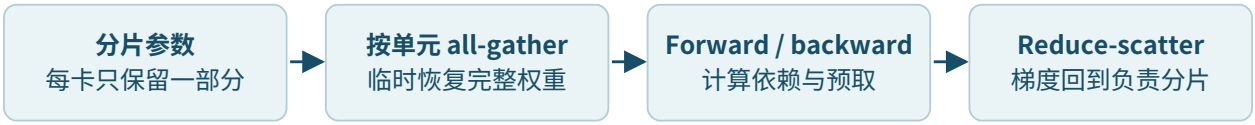
分片单元太大，瞬时全参数gather造成峰值；太小则频繁小collective及调度开销。将一层作为单元是起点而非定律。pre-fetch下一层有机会隐藏通信，但会让两层参数同时驻留，显存预算必须包含这部分。

实现方案与验证

在目标版本验证mixed precision、CPU offload、activation checkpoint、torch.compile和checkpoint格式支持。先做一层多卡数值回归，再全模型。测memory peak和通信overlap，改变wrap粒度时同时检查collective顺序与反向生命周期。

常见追问与易错点

追问：FSDP一定比DDP更快吗？答：它优先解决容量和冗余，额外gather可能更慢；容量允许的DDP仍有价值。追问：FSDP2就是换一个函数名吗？答：参数表示和使用方式有变化，需按版本迁移，不应机械替换。



最大 materialized 单元 + 预取窗口 会显著影响峰值显存。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T10] [T33] [T08] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 16

16 CPU/NVMe offload 与 Hybrid Sharding 如何选?

P1 · 进阶补充 公开记录同类主题：[I06] [I13]

> **面试口述版**
>
> offload把显存压力转移到CPU内存或存储，并增加搬运与CPU计算成本；hybrid sharding常在高速域内分片、域间复制/同步，以控制慢网络上的频繁参数聚合。它们都在容量、带宽和并发之间交换，不是无成本省显存。

背景：问题为什么会出现

优化器offload可将状态及更新留在CPU；参数/激活offload更依赖PCIe/NVLink-C2C等路径和异步prefetch。NVMe多一层存储延迟。是否受益需要比较搬运时间与可重叠的计算时间，而非只看CPU还有多少内存。

原理与推导：把关键条件说清楚

教学例子：每步需要额外搬运20GB，若持续有效带宽25GB/s，单次方向传输下界约0.8s；若可隐藏的计算仅0.2s，大量开销仍暴露。此数值是人工假设，不是某机器基准。双向、NUMA与共享链路会让实际更复杂。

实现方案与验证

先测H2D/D2H与CPU optimizer，不在完整训练里盲试大量offload参数。使用pinned memory、预取与生命周期管理时要限制系统内存压力；把页错误、NUMA、I/O queue和网络一同纳入监控。容量能满足时优先减少不必要offload。

常见追问与易错点

追问：分片组越大越省显存就越好？答：跨慢链路的gather可能主导step。追问：offload可以解决activation爆炸吗？答：只有搬运/重计算相应激活才行，仅optimizer offload不改变attention中间张量。

技术依据：[T08] [T33] [T21] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 17

17 TP 的列并行和行并行，前向与反向各通信什么?

P0 · 优先掌握 公开记录同类主题：[I05] [I10]

> **面试口述版**
>
> 列并行按输出维切W，输入X可复制，各rank产生一部分输出；行并行按输入/归约维切W和X，各rank得到输出partial sum，再归约。把FFN的第一层列并行与第二层行并行搭配，可在中间非线性处保留分片，减少不必要通信。

背景：问题为什么会出现

采用Y=XW、W=[K,N]约定：列切N，Y=[Y0,Y1,…]；行切K，Y=Σ_i XiWi。PyTorch Linear.weight存储通常是[N,K]，讨论“行列”必须先说明矩阵约定，否则容易与源码维度相反。

原理与推导：把关键条件说清楚

列并行反向：各rank计算本地dWi，输入梯度dXi=dYiWi^{T}需按TP组求和；行并行前向对partial Y求和，反向若上游dY已复制，各rank直接算本地dXi和dWi。实际SP/融合可把all-reduce变成reduce-scatter/all-gather，逻辑语义仍须保持。

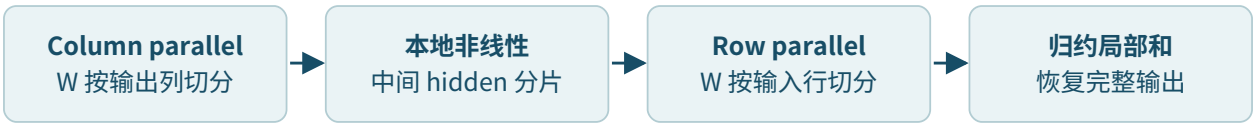
```
Column: W=[W0 W1], Y=[XW0 XW1] ; Row: X=[X0 X1], W=[W0;W1], Y=X0W0+X1W1
```

实现方案与验证

附录用两分片矩阵重建forward与gradient，CPU验证与完整GEMM一致。GPU上查看激活大小、collective频率、拓扑与小M退化。TP增加也减少单卡GEMM尺寸，不能只看参数被切小。

常见追问与易错点

追问：为什么不两层都列并行？答：第二层需要消费第一层完整输入或改布局，可能增加gather。
追问：TP=8就快8倍吗？答：通信、形状效率与复制计算共同限制。



配对可避免不必要的中间 all-gather；实际 forward/backward 通信需按布局推导。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T11] [T12] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 18

18 Attention 的 TP 怎么切？GQA 的 KV heads 少于 TP 时怎么办？

P0 · 优先掌握 公开记录同类主题：[I10] [I05]

> **面试口述版**
>
> 常见做法按head切QKV投影及attention计算，输出投影沿输入维切后归约。GQA的query heads多、KV heads少，TP度数超过KV heads时不能简单平均切每个head，可能需要KV复制或特定分组映射。

背景：问题为什么会出现

先列hq、hkv、dh和TP，检查整除约束与同一KV head服务哪些query heads。QKV融合权重布局可能按head交错或分块，权重重分片必须与kernel预期一致。

原理与推导：把关键条件说清楚

例如hq=32、hkv=8、TP=4时每rank可有8个Q heads、2个KV heads。若TP=16，不能说每rank有0.5个KV head；实现可能复制KV或用其他布局。这会改变每卡权重/KV与通信量，也意味着“KV显存除TP”不是无限成立。

实现方案与验证

以实际模型配置和框架支持为准，建立TP=1与多TP的logits/gradient对照。检查RoPE维度、head mapping、cache布局与输出投影。训练和rollout的TP不同还会牵涉权重布局同步，不能只通过shape相同判断正确。

常见追问与易错点

追问：head_dim可以任意变化来凑整除吗？答：那通常改变模型结构/权重语义，不是无损部署技巧。追问：Attention算量是否全可按TP均分？答：理想主GEMM可分，部分norm、mask、同步与复制KV不一定。

技术依据：[T01] [T11] [T12] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 19

19 PP 的 GPipe、1F1B、microbatch 与 bubble 如何计算?

P0 · 优先掌握 公开记录同类主题：[I06] [I10]

> **面试口述版**
>
> PP按层切模型，microbatch在stage之间流水。GPipe通常先多批前向再反向，1F1B在warmup后交替前反向，减少待反传激活数。bubble受stage数、microbatch数、负载不均和通信影响，不是固定一个百分比。

背景：问题为什么会出现

在stage完全均衡、前反向时间固定、同步flush且忽略通信的理想模型中，p个stage、m个microbatch，bubble占总时长约(p−1)/(m+p−1)。常见(p−1)/m是相对有效工作时间的开销比，不是同一分母。

原理与推导：把关键条件说清楚

例p=4、m=8，理想bubble fraction=3/11≈27.3%，而开销/有效时间=3/8=37.5%。增加m能降低bubble，但若global batch固定，microbatch变小可能让GEMM效率变差；更多microbatch也增加调度和通信启动。

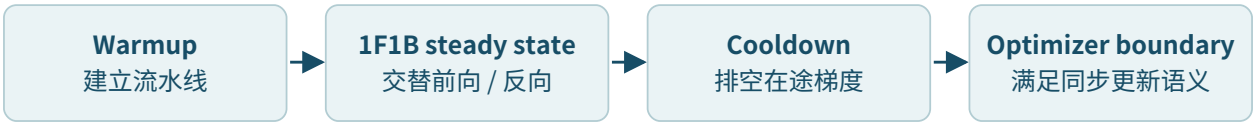
```
bubble_fraction ≈ (p−1)/(m+p−1) ; overhead/useful ≈ (p−1)/m
```

实现方案与验证

先按实际F/B耗时而非层数平均分stage，embedding/lm_head和MoE层常不均。画时间线检查stage idle与链路传输。比较GPipe和1F1B时同时测峰值激活，保持有效batch、loss归一化和更新时点一致。

常见追问与易错点

追问：1F1B一定无bubble吗？答：仍有warmup/cooldown和负载差。追问：每stage更新自己的权重能消除等待吗？答：会产生权重版本不一致，属于不同调度算法，不能当作同步PP的等价实现。



更多 microbatches 减少理想 bubble，但会增加调度、通信与小 GEMM 成本。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T34] [T12] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 20

20 Interleaved pipeline、virtual stage 与 zero-bubble 思路是什么?

P1 · 进阶补充 公开记录同类主题：[I06] [I10]

面试口述版

虚拟stage把每张设备上的层拆成多个模型块，允许更细粒度流水交错，降低部分bubble，但会增加通信次数和调度复杂度。zero-bubble一类方法尝试将反向拆成dgrad与wgrad，用非关键的wgrad填空隙，并非真实系统永远零空闲。

背景：问题为什么会出现

下游stage继续反向通常急需dX，上游权重更新则还需dW，两者有不同依赖。拆开后可推迟某些wgrad来填补空洞，但必须在optimizer step前全部完成，并保存或重建所需激活。

原理与推导：把关键条件说清楚

一个调度优化是否有效，要同时满足依赖正确、显存预算、通信进度与stage balance。virtual stage越多，单块计算越短，对链路和CPU调度更敏感；小microbatch时每次传输的启动成本可能抵消bubble收益。

实现方案与验证

实现前画DAG，把每条激活和梯度的产生/消费者标清，尤其recompute与共享权重。使用固定batch验证梯度与标准同步参考一致，再以trace比较关键路径。不要只按图上空格数量估算实际速度。

常见追问与易错点

追问：wgrad能全部延到最后吗？答：可能增加保存激活和尾部计算，且必须满足更新前完成。追问：优化理论bubble后MFU仍低？答：GEMM小、通信争用、数据等待或重算可能成为新瓶颈。

技术依据：[T12] [T34] [T38] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 21

21 SP、CP、Ulysses、Ring Attention 分别切什么、传什么?

P0 · 优先掌握 公开记录同类主题：[I05] [I10]

> **面试口述版**
>
> 术语必须放在框架语境中。Megatron常见SP在TP组内分摊norm/dropout等序列激活；CP让attention序列也跨设备分布；Ulysses用all-to-all在序列切分与head切分之间转换；Ring通过传递KV块让本地Q逐块完成attention。

背景：问题为什么会出现

长序列使激活和attention成为瓶颈。仅把norm按序列分片，不会自动让所有attention操作都处理更短序列；这正是SP与完整CP需要区分的地方。不同文献也可能用sequence parallel统称，不能只看名字。

原理与推导：把关键条件说清楚

Ulysses交换QKV布局，让每rank获得较少head的长序列；因此head数、GQA映射和并行度存在约束。Ring保持Q块，在多个通信轮次取得KV，使用在线softmax合并局部统计；可与计算重叠，但带宽/块大小和causal负载不均会影响效率。

实现方案与验证

按[batch,sequence,head,head_dim]画每rank张量；明确每次all-to-all或P2P的输入输出字节。验证causal mask、位置编号、dropout RNG以及backward的KV梯度归约。不要把推理DCP的Q通信/局部O+LSE合并直接当训练CP全部流程。

常见追问与易错点

追问：CP能代替TP吗？答：解决的主要维度不同，可组合；CP并不直接按比例减少模型参数。追问：Ring能无限长且免费吗？答：总计算、通信和KV/激活仍增长，受硬件与实现约束。

路线	切分与计算	通信要点
Ulysses	sequence shard → head shard	All-to-All 交换 Q/K/V
Ring / CP	Q 本地；KV 按块流动	在线 Softmax / O-LSE 合并

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T13] [T14] [T15] [T11] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 22

22 MoE 训练中 EP、dispatch、combine、capacity 与负载均衡怎么解释?

P0 · 优先掌握 公开记录同类主题：[I10] [I13]

> **面试口述版**
>
> Router给token选Top-k专家；dispatch将它们重排并发到专家所在rank；专家做FFN；combine按路由权重返回合并。EP减少每rank持有的专家数，但引入all-to-all和负载不均；参数总量、激活参数和通信量是三本账。

背景：问题为什么会出现

设T个token、E个专家、每token选k个，平均每专家路由量约Tk/E，但真实分布可能长尾。capacity与padding决定缓冲区与GEMM布局；drop token、dropless或动态容量是不同语义，不能把溢出都当性能细节。

原理与推导：把关键条件说清楚

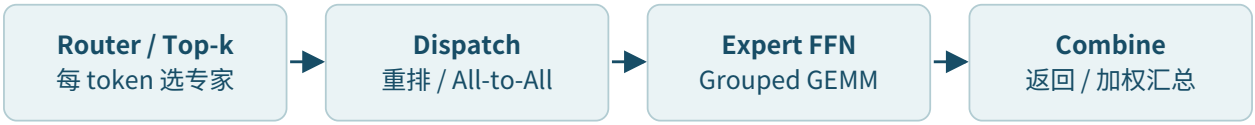
粗略每次跨设备传activation的上界规模与T · k · d · bytes相关，但同rank专家去重、节点内转发、共享专家和通信dtype会改变实际量。训练还包括梯度的反向通信，不能只用forward dispatch带宽预测整步速度。

实现方案与验证

记录每专家/每rank token直方图、padding比例、被丢弃量、dispatch/GEMM/combine时间和最长rank。负载均衡loss或路由bias调整要看最终质量，不能为了完全均匀破坏模型选择。先控制路由分布做kernel基准，再用真实训练trace验证。

常见追问与易错点

追问：DeepEP/通信后端替换等于整个MoE优化吗？答：还包括重排、grouped GEMM、量化、共享专家与反向。追问：router replay解决负载不均吗？答：它重用路由路径，不是自动负载均衡算法。



总专家数 E 决定参数容量；激活数 k、负载偏斜和拓扑决定实际执行成本。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[R21] [T11] [T29] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 23

23 AllReduce、ReduceScatter、AllGather、AlltoAll 的区别与通信量?

P0 · 优先掌握 公开记录同类主题：[I05] [I06]

> **面试口述版**
>
> AllReduce先归约并让所有rank得到同一结果；ReduceScatter将归约结果分片；AllGather汇集各rank分片；AlltoAll为每个目标rank发送不同数据。它们是通信语义，不等同于唯一的ring或tree实现。

背景：问题为什么会出现

对W个rank、每rank输入完整S bytes的理想ring AllReduce，reduce-scatter与all-gather各发送(W−1)/W · S，总发送2(W−1)/W · S；接收同量。如果把发送+接收一起统计会再乘2，必须声明口径。

原理与推导：把关键条件说清楚

例W=8、S=1GiB，单rank发送约1.75GiB，接收约1.75GiB。该公式不包含协议、拓扑转发和分层算法额外代价；AllGather中若S指“每rank本地分片”而非最终总量，公式又不同。面试最容易错在S的定义。

$$V_{send,ring\text{-}AR} = 2(W-1)S/W \; ; \; t \approx n_{steps}\cdot\alpha + V_{send}/BW_{eff}$$

实现方案与验证

用α延迟+bytes/带宽模型解释小包为何偏启动延迟、大包为何偏带宽，再用nccl-tests校准。所有rank须遵守匹配的collective序列、count/dtype等要求；某rank先退出可能使其他rank报通信超时。

常见追问与易错点

追问：AllReduce一定等于两次独立API调用吗？答：语义可分解为RS+AG，但实现可融合/分层。追问：AlltoAll需要每rank发相同字节吗？答：固定count接口与变长交换有不同约定，要看具体API与split元数据。

操作	语义	典型用途
AllReduce	Reduce-scatter + All-gather	全量结果回到各 rank
Reduce-scatter	先归约，再各取一片	常用于梯度分片
All-to-All	每个 rank 发不同分片	常用于 EP / Ulysses

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T16] [T12] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 24

24 GPU 拓扑、NVLink、PCIe、NUMA、RDMA 对并行布局有什么影响?

P0 · 优先掌握 公开记录同类主题：[I05] [I06]

> **面试口述版**
>
> 将高频细粒度通信尽量放在高带宽低延迟域内，将相对粗粒度的并行放到较慢域，是合理起点，但必须由真实拓扑和profile验证。机器数量、NUMA与GPU互联域不能简单画等号。

背景：问题为什么会出现

TP每层通信频繁，EP受token all-to-all分布影响，DP梯度通信有较大buffer并可能重叠，PP以stage边界P2P为主。NIC到GPU的亲和性、CPU pinning与PCIe路径可能决定有效带宽。

原理与推导：把关键条件说清楚

拓扑表里CPU跨NUMA不自动意味着GPU间一定绕CPU；同NVSwitch域的GPU可能仍通过NVLink互通。反过来GPU型号相同也不保证每对链路一样。跨机RDMA还涉及GDR、网络路由、拥塞与交换机超售。

实现方案与验证

收集nvidia-smi topo -m、GPU/NIC PCIe信息、NUMA绑定、NCCL topology/log；先测同节点P2P，再测跨节点小包/大包collectives。优先修正拓扑、接口和权限问题，再扫NCCL算法环境变量；保留干净默认配置作对照。

常见追问与易错点

追问：NVLink总带宽数字能直接代入AllReduce时间吗？答：不能，单双向口径、消息尺寸、算法和共享链路影响持续有效带宽。追问：跨NUMA就是慢？答：CPU访问和GPU互联要分别分析。

技术依据：[T17] [T11] [T26] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 25

25 通信计算 overlap 为什么“开了 async”仍可能没加速?

P0 · 优先掌握 公开记录同类主题：[I05] [I06]

> **面试口述版**
>
> 异步提交只说明CPU无需立即等待，不证明GPU执行有真实重叠，更不保证关键路径缩短。通信与计算可能争用SM、HBM、PCIe/NVLink，或者被stream依赖、bucket就绪时机和buffer生命周期串行化。

背景：问题为什么会出现

理想情况下独立计算C与通信M串行耗时C+M，重叠下界max(C,M)。但若通信占用资源使计算变慢为C′，或存在依赖尾巴M_tail，真实时间并不等于下界。

原理与推导：把关键条件说清楚

例如计算8ms、通信6ms，理想可降到8ms；若重叠后GEMM因资源争用变11ms，则收益只剩3ms，甚至在更差配置下倒退。AllGather prefetch需要提前知道下一层，并保证聚合参数的显存峰值；梯度通信要等对应bucket就绪。

```
t_serial=C+M ; t_overlap,ideal≥max(C,M)
```

实现方案与验证

用Nsight Systems/PyTorch Profiler同时看stream时间线与step边界；在隔离实验中测GEMM单跑、通信单跑、并发跑。调整bucket、prefetch距离、comm SM或融合粒度时，同时报告关键路径和内存，避免只报“overlap比例”。

常见追问与易错点

追问：看到两个stream颜色重叠就成功了吗？答：还需看实际kernel、资源竞争与总step时间。追问：CPU非阻塞拷贝一定异步H2D吗？答：涉及pinned memory、stream和硬件能力，需要实测。

技术依据：[T09] [T33] [T25] | 口述、数值例子及实验建议为本册组织与推导。

03 分布式与通信 / QUESTION 26

26 多维并行 world size 如何计算？EP 为什么不能总额外乘进去？

P0 · 优先掌握 公开记录同类主题：[I10] [I13]

> **面试口述版**
>
> 先画process mesh并说明每个维度是否独立。常见稠密布局world=TP×PP×CP×DP；MoE的EP可能复用或重排原DP相关维度，而不是无条件再乘一个EP。不同子模块还可能采用不同mesh。

背景：问题为什么会出现

如果框架把DP组中的rank进一步组织成EP组，增加EP表示重新分配专家，不一定增加物理卡数。expert tensor parallel与dense TP也可能不同。简单“五个数字相乘”会错误预算设备和梯度同步组。

原理与推导：把关键条件说清楚

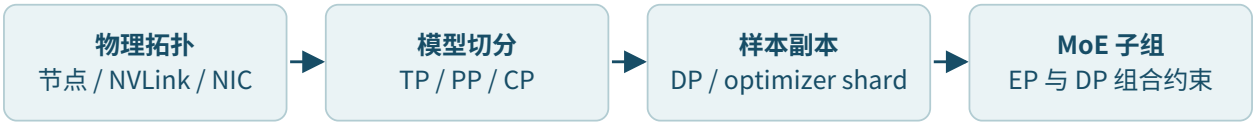
教学例：world=64、TP=4、PP=2、CP=1，则剩余DP=8；若在该DP维度划EP=4，仍是64卡，专家副本维度可与原稠密DP不同。这个示例只说明一种mesh组织，实际Megatron/框架约束须查配置和group生成代码。

实现方案与验证

打印每rank的global/local rank、TP/PP/CP/DP/EP组成员，用小world验证通信范围。全局batch按独立数据副本计算；shared参数与expert参数的归约组不能混。保存checkpoint时也要记录mesh，重启重分片不能靠rank号猜参数含义。

常见追问与易错点

追问：EP和DP同时用时专家梯度同步给谁？答：给持有同一专家副本的相应数据并行组，不一定是所有dense DP rank。追问：配置能启动就对吗？答：错误group也可能运行但更新语义不对，需梯度参考验证。



并行组是 rank 集合与映射；不要把所有缩写机械相乘。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T11] [T12] [R21] | 口述、数值例子及实验建议为本册组织与推导。

04 算子、内存与编译 / QUESTION 27

27 梯度累积遇到不同长度，怎样保证全局 token mean 正确?

P0 · 优先掌握 公开记录同类主题：[I12] [I16]

> **面试口述版**
>
> 不要平均“平均值”。目标是整个有效batch的token loss总和除以有效token总数；不同microbatch/rank长度不同时，本地mean再平均会改变权重。应按分母统一后反传，或使用明确的样本/序列加权目标。

背景：问题为什么会出现

设第r个DP rank第j个microbatch的损失和S_rj、有效计数N_rj，目标L=Σ_rj S_rj/Σ_rj N_rj。若后续梯度默认除DP大小W，则每个microbatch本地反传W · S_rj/N_global，并在累计完成后只step一次。

原理与推导：把关键条件说清楚

如果N_global在训练前已知，可预先从labels/masks统计；否则需要一个与框架兼容的重新缩放策略，注意scaler、通信与optimizer语义。不能在每个rank用各自计数代替全局计数，也不能把loss日志all-reduce当作修好梯度。

```
N_global=Σ_{r,j}N_rj ; L_local,rj = W·S_rj/N_global
```

实现方案与验证

附录通过把同一线性CE样本拆成不等长shard验证梯度等价。再加入梯度累积、全paddingrank、最后不足batch、CP切分等场景，明确哪些rank负责哪个token，以防重复计数。归一化调整应先CPU参考再GPU验证。

常见追问与易错点

追问：固定样本mean目标怎么办？答：先给每样本定义loss再按样本权重归约，这是另一种合法目标，不能和token mean混称等价。追问：每microbatch裁剪会影响吗？答：会，clip非线性，通常累计完成后做。

技术依据：[T09] [T05] | 口述、数值例子及实验建议为本册组织与推导。

04 算子、内存与编译 / QUESTION 28

28 Activation checkpoint 的原理、选择性重算与 RNG 风险?

P0 · 优先掌握 公开记录同类主题：[I06] [I13]

> **面试口述版**
>
> Checkpoint不保存某段前向的全部中间激活，反向时根据保留的输入重新计算，用算力换显存。选择性重算挑选内存大、重算便宜的部分，但要保证重新执行的函数在相关状态和随机性上与原前向一致。

背景：问题为什么会出现

它不是训练checkpoint文件。重算过程中dropout、随机路由、全局计数器、in-place或外部状态变化都可能让反向对应另一条函数。是否preserve RNG、reentrant/non-reentrant的差异与支持边界要按PyTorch版本核对。

原理与推导：把关键条件说清楚

融合FlashAttention自身也可能在backward重算局部概率；再外包一层checkpoint，会叠加计算。全层重算易接入但成本大；selective策略可保留昂贵GEMM结果、重算部分便宜逐元素或归约，需要以实际保存张量与算子实现判断。

实现方案与验证

比较同batch的loss/grad与无checkpoint参考，覆盖dropout、MoE路由和多次调用。记录峰值allocated、F/B耗时、通信重叠变化。若重算触发额外参数all-gather，还要计入FSDP交互成本，而不是只数多跑了几个GEMM。

常见追问与易错点

追问：checkpoint后forward耗时不变，为什么backward慢？答：重算主要在反向阶段触发。追问：只要seed一样就够？答：还要保持随机数流、执行次序与状态；并行路由重放可能需要额外记录。

技术依据：[T20] [T05] [T18] | 口述、数值例子及实验建议为本册组织与推导。

04 算子、内存与编译 / QUESTION 29

29 FlashAttention 与 Online Softmax 如何推导？LSE 怎么合并？

P0 · 优先掌握 公开记录同类主题：[I05] [I07]

> **面试口述版**
>
> FlashAttention以tile计算QK^T和AV，不把完整attention矩阵写入HBM；在线softmax维护每行最大值、归一化分母和加权值和，遇到新block时重缩放旧统计。它保留精确attention语义但改变访存与浮点执行顺序。

背景：问题为什么会出现

对每块scores S_j，维护m=max分数、l=Σexp(S−m)、u=Σexp(S−m)V。新最大值m′=max(m,maxS_j)，旧l和u乘exp(m−m′)，再加入新块项；最后O=u/l、LSE=m+log l。关键不是逐块各自softmax后直接相加。

原理与推导：把关键条件说清楚

若已有局部归一化输出O_i和LSE_i，合并L=logsumexp(LSE_i)，O=Σexp(LSE_i−L)O_i。例两个分区LSE为log2、log3，输出2、8，则总O=(2・2+3・8)/5=5.6，而不是(2+8)/2。

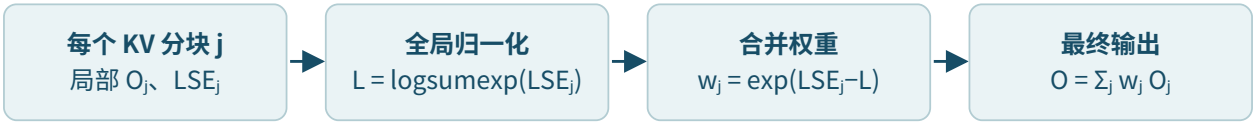
```
L=logsumexp_i(LSE_i) ; O=Σ_i exp(LSE_i−L)·O_i
```

实现方案与验证

测试单块/多块、分区顺序、极端logits、causal边界和全mask行。全mask时LSE可能为−∞，必须避免−∞−(−∞)产生NaN。backward通常根据Q/K/V与保存统计重算局部P，再计算dQ/dK/dV；需验证梯度而不只是O。

常见追问与易错点

追问：FlashAttention把复杂度降到线性吗？答：主要减少HBM中间存储/访问，稠密注意力计算仍二次。追问：LSE用log2还是ln有区别吗？答：合并指数基必须匹配实现，不能直接混用。



局部 O 已归一化，不能直接相加；全 mask 分块需要显式边界处理。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T18] [T15] | 口述、数值例子及实验建议为本册组织与推导。

04 算子、内存与编译 / QUESTION 30

30 Register、shared memory、occupancy 与 persistent pipeline 怎么权衡?

P1 · 进阶补充 公开记录同类主题：[I05] [I07]

> **面试口述版**
>
> 每CTA的寄存器、shared memory、线程和架构上限共同限制驻留CTA/warp。更高occupancy不必更快；较大tile与更深流水可增加复用、隐藏延迟，却占更多资源。persistent kernel减少重复调度并在核内分配工作，但仍需足够的独立任务和软件流水。

背景：问题为什么会出现

粗略驻留数受min(register_budget/reg_per_CTA,smem_budget/smem_per_CTA,thread_limit/threads_per_CTA,CTA_limit)约束，并且存在分配粒度/架构细则。不能只拿每线程register数除一次就断言真实occupancy。

原理与推导：把关键条件说清楚

num_stages增加让访存与MMA更可重叠，但SMEM buffer也变大；寄存器压力高会spill到local memory，实际走HBM/L2路径而非免费片上寄存器。Hopper/Blackwell的异步copy、MMA、TMEM各有资源和同步规则，不能用更多stages一把梭。

实现方案与验证

先确认bound是内存、算力还是依赖等待，扫tile/stages/warps时记录寄存器、SMEM、spill、stall与耗时。TMA适合描述符驱动的规则tensor搬运；cp.async类路径可适应不同粒度，选择需结合对齐/形状/复用而非“新指令一定更快”。

常见追问与易错点

追问：persistent只驻留一个CTA还能隐藏延迟吗？答：可能依赖warp specialization、双缓冲与指令级并行，但不是自动做到。追问：SM100与SM120能套同一资源数字吗？答：不能，Blackwell也有不同计算能力与资源上限。

技术依据：[T26] [T30] [T31] | 口述、数值例子及实验建议为本册组织与推导。

04 算子、内存与编译 / QUESTION 31

31 Grouped GEMM 的 contiguous / masked 布局及 tile scheduling?

P1 · 进阶补充 公开记录同类主题：[I07] [I13]

面试口述版

Grouped GEMM把多个专家的小矩阵乘合并调度。contiguous常把各expert的有效token按连续段组织并按tile对齐；masked常在固定容量布局上用有效长度/掩码限制计算。具体术语必须对应所用库，不是所有实现的统一格式。

背景：问题为什么会出现

MoE每expert的M动态变化，N/K常固定。逐expert启动小GEMM会有launch与占用问题；统一tile调度可分摊长短任务，但仍需知道每个tile属于哪个expert及其有效行数。

原理与推导：把关键条件说清楚

示例三个专家有效M=[20,130,0]，M-tile=64。连续padding后可能是[64,192,0]，共4个M tiles；若每expert都预留192容量则固定布局浪费更大，masked可跳无效部分但要计调度/元数据。此为示意，具体padding不能从算法名猜。

实现方案与验证

观察路由分布、tiles数、padding比例、per-expert权重L2复用和尾部rank。swizzle改变tile访问顺序以改善局部性，不改变数学结果；persistent scheduler还要避免大专家饿死小专家或使负载更不均。验证zero-token、尾tile与wgrad累积。

常见追问与易错点

追问：masked一定比contiguous适合decode吗？答：常见如此，但取决于容量、固定shape需求与实现；训练也可用masked。追问：只测均匀随机路由够吗？答：不够，热点和极端偏斜可能主导真实性能。

技术依据：[T29] [R21] | 口述、数值例子及实验建议为本册组织与推导。

04 算子、内存与编译 / QUESTION 32

32 torch.compile、算子融合与 CUDA Graph 分别解决什么?

P1 · 进阶补充 公开记录同类主题：[I07] [I13]

> **面试口述版**
>
> torch.compile尝试捕获与优化计算图并生成更高效执行；融合减少launch与中间读写；CUDA Graph捕获可重复的GPU执行序列以降低CPU提交开销。三者可配合，但CUDA Graph本身不负责消除所有算法冗余或自动融合所有kernel。

背景：问题为什么会出现

动态shape、Python控制流、tensor.item()、动态分配与通信生命周期可能影响图捕获或产生graph break。训练还需覆盖backward、optimizer和RNG，不能把推理graph经验直接照搬。

原理与推导：把关键条件说清楚

以小kernel密集的step为例，GPU间隙来自CPU launch时CUDA Graph有意义；若主要时间在大GEMM或跨机通信，launch优化的上限有限。shape bucketing可减少编译/graph变体，但padding本身也有额外计算。

实现方案与验证

先建立eager数值与速度基线，分开统计首次compile/capture和warm稳态；检查graph break/recompile次数、内存池峰值以及不同shape分布。捕获前完成需要的初始化，重放时维持地址/状态约束；最终以完整step而非单kernel测量。

常见追问与易错点

追问：compile后更慢怎么办？答：排查编译缓存、动态重编译、融合造成的寄存器压力和缺失的高效库路径。追问：forward能capture代表训练能吗？答：不代表，backward/optimizer/collective都需分别验证。

技术依据：[T21] [T05] [T39] | 口述、数值例子及实验建议为本册组织与推导。

05 数据、稳定性与恢复 / QUESTION 33

33 数据 pipeline、tokenization、packing、mask 如何影响训练质量和吞吐?

P0 · 优先掌握 公开记录同类主题：[I11] [I12] [I16]

> **面试口述版**
>
> 训练数据管线既要供得上GPU，也要保证目标正确。tokenizer/chat template/去重/截断/packing都会改变模型看到的上下文与标签；packing减少padding，但独立样本间的attention可见性和position语义必须明确。

背景：问题为什么会出现

把多条短样本直接拼起来并不自动等于独立训练。如果任务要求跨样本隔离，需要block-diagonal或等价varlen attention边界；SFT loss mask也要按每条对话重建。允许跨文档上下文则是另一种明确的数据配方。

原理与推导：把关键条件说清楚

吞吐瓶颈可能在远端存储、解压、tokenization、Python collate或H2D。增加num_workers不一定线性加速，可能带来重复数据、内存压力、NUMA跨节点访问和随机种子问题。prefetch和pinned memory同样需要预算。

实现方案与验证

先用合成已tokenized数据测GPU上限，再逐项恢复真实I/O，记录data wait。对每条packed样本保存segment边界，测试shift、EOS、ignore_index和跨segment注意力。DistributedSampler设置epoch；可迭代数据源需显式按rank/worker分片防重复。

常见追问与易错点

追问：数据越多越好吗？答：质量、重复与目标分布同样重要。追问：训练resume只恢复shuffle seed足够吗？答：还要恢复epoch/游标/worker或stream状态，否则会重读或跳过样本。

技术依据：[T24] [T01] [T05] | 口述、数值例子及实验建议为本册组织与推导。

05 数据、稳定性与恢复 / QUESTION 34

34 LoRA 与 QLoRA 省哪些显存？冻结基座是否可以 no_grad?

P0 · 优先掌握 公开记录同类主题：[I07] [I12]

> **面试口述版**
>
> LoRA把可训练增量写成低秩BA，只为小矩阵维护梯度/优化器；基座权重仍要参与前向和必要的输入梯度。QLoRA进一步以量化形式保存冻结基座，但激活与计算不等于全4bit，不能把整个训练显存按权重位宽缩放。

背景：问题为什么会出现

对于W=[d_out,d_in]，A=[r,d_in]、B=[d_out,r]，增量参数r(d_in+d_out)，少于d_out · d_in；实际还可能有scale=α/r等约定。初始化常让增量初始为零，保证起点与基模一致。

原理与推导：把关键条件说清楚

例4096×4096矩阵、r=16，LoRA参数131072，占原16777216参数约0.78%。但输入activation、attention、logits和基座权重依然在；若要将loss传到前层的adapter，冻结线性层也需计算dX，不能全部包no_grad。

$$W' = W + (\alpha/r)BA \ ; \ P_{LoRA} = r(d_{in} + d_{out})$$

实现方案与验证

区分训练adapter、merge后的部署权重、量化后merge路径和多adapter热切换。测试初始forward等价、基座梯度为None、adapter有梯度、merge/unmerge误差。QLoRA需包括量化scale/metadata和dequant工作区，报告完整质量与时间。

常见追问与易错点

追问：LoRA秩越大越好？答：表示能力与过拟合/成本权衡，依数据和目标模块。追问：训练完一定无额外推理成本？答：可合并时通常可避免adapter额外matmul，但量化或动态多adapter部署有不同路径。

技术依据：[T27] [T28] [T05] | 口述、数值例子及实验建议为本册组织与推导。

05 数据、稳定性与恢复 / QUESTION 35

35 OOM、显存持续增长与 fragmentation，怎么定位而不是乱减 batch?

P0 · 优先掌握 公开记录同类主题：[I06] [I13]

> **面试口述版**
>
> 先确定峰值发生在哪一步，再区分真实存活张量、缓存allocator保留、外部库内存和碎片。持续增长更像引用/图/缓存未释放，固定点OOM更像容量或瞬时工作区；二者不应都用empty_cache处理。

背景：问题为什么会出现

allocated近似活跃tensor分配，reserved是缓存allocator向设备保留的池，nvidia-smi还含context和部分外部分配。reserved大于allocated本身不证明泄漏，也不证明所有差额都能满足下次大块分配。

原理与推导：把关键条件说清楚

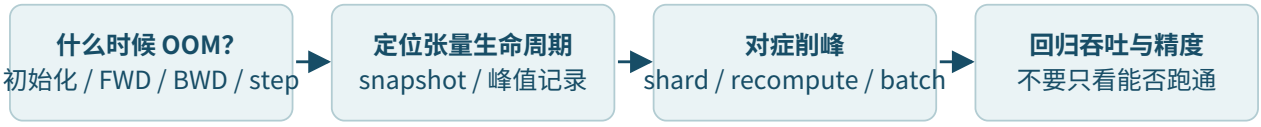
常见泄漏：把loss tensor/输出保存到list，hook捕获历史图，retain_graph长期打开，重复创建CUDA Graph池，动态shape缓存无上限。FSDP gather、optimizer首次创建状态和checkpoint materialization也会制造一次性峰值。

实现方案与验证

在阶段边界reset并记录peak memory，采集memory snapshot，按固定batch重复运行看曲线。用最小复现逐个移除日志、eval和缓存。先修复引用，再考虑checkpoint、分片、microbatch、offload或allocator设置；任何环境变量都保留前后证据。

常见追问与易错点

追问：empty_cache为什么有时看似有效？答：释放未使用缓存或改变分配状态，不会释放被引用的tensor；频繁调用还可能降低性能。追问：OOM后跳过当前batch继续行吗？答：分布式各rank必须同步恢复策略，否则可能挂在collective。



改 allocator 或 empty_cache 不能释放仍被活跃张量占用的显存。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T21] [T25] [T05] | 口述、数值例子及实验建议为本册组织与推导。

05 数据、稳定性与恢复 / QUESTION 36

36 Loss spike、NaN、梯度爆炸如何做第一现场排查?

P0 · 优先掌握 公开记录同类主题：[I06] [I13]

> **面试口述版**
>
> 先保存触发batch与前一checkpoint，定位第一个出现非有限值的张量/算子，而不是看到最后loss NaN就调学习率。常见源包括坏输入、全mask softmax、除零、FP16溢出、错误归约和自定义kernel越界。

背景：问题为什么会出现

将故障按“首次forward就坏、backward才坏、step后才坏、多卡才坏、运行久才坏”分类，可大幅缩小范围。优化器state损坏会让正常forward梯度在step后把权重写坏；loss scaling跳步日志也应检查。

原理与推导：把关键条件说清楚

最小重放从高精度或eager开始，关闭新融合/量化，再逐层恢复。检查logsumexp稳定性、norm eps、padding/causal边界与reward尺度。异常检测可帮助找反向调用栈但会拖慢运行，不宜把debug配置当性能基准。

实现方案与验证

记录数据ID、模型/optimizerstate、RNG、代码commit、各rank首个错误时间。对自定义CUDA算子加入reference、sanitizer和边界shape。修复后做固定样本回归和长跑，不仅“下一步没NaN”。

常见追问与易错点

追问：grad clipping能治所有loss spike吗？答：不能，输入/标签错误和forward数值已经坏时无效。追问：BF16就不溢出吗？答：动态范围大不等于任何数学操作都安全。

技术依据：[T07] [T05] [T23] | 口述、数值例子及实验建议为本册组织与推导。

05 数据、稳定性与恢复 / QUESTION 37

37 NCCL timeout/hang 怎么查？为什么不先把 timeout 调大？

P0 · 优先掌握 公开记录同类主题：[I06] [I13]

> **面试口述版**
>
> 通信超时经常是症状：某rank先OOM、数据加载阻塞、CUDA异常或控制流不一致，使其他rank等不到匹配collective。先找最早异常及collective序列，再检查网络和拓扑，调大timeout只可能延后报错。

背景：问题为什么会出现

必要条件包括所有参与rank对同一group以匹配顺序调用collective，以及一致的count/dtype等约定。动态跳过loss、某rank提前结束dataloader、checkpoint仅部分rank执行，都可能造成分布式死锁。

原理与推导：把关键条件说清楚

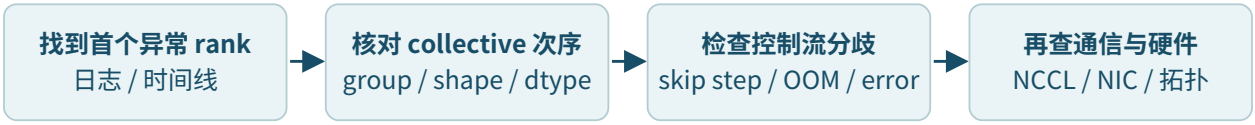
分类路径：单机也失败→先查程序顺序/设备异常；跨机才失败→查NIC、GDR、IB/RoCE与网络连通；只大消息慢→看带宽/拥塞；运行随机挂→看rank首错、内存、网卡和硬件日志。不要同时修改十个NCCL环境变量而失去因果。

实现方案与验证

保留所有rank的带时间戳日志，启用适量NCCL diagnostics，使用框架支持的trace/flight recorder。用同规模nccl-tests测基础设施，再用极简训练检查应用层。任何barrier都不是万能修复，它可能只是把死锁转移到另一行。

常见追问与易错点

追问：只有rank0报timeout，能认定rank0故障吗？答：不能，最先打印的等待方可能正常。追问：NCCL_DEBUG=INFO会解决问题吗？答：提供证据，是否解决取决于真实原因。



其他 rank 的 timeout 常是后果；先定位最早出错或走偏的进程。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T17] [T16] [T09] ｜ 口述、数值例子及实验建议为本册组织与推导。

05 数据、稳定性与恢复 / QUESTION 38

38 设随机种子就确定性了吗？如何验证训练优化没有改变结果？

P0 · 优先掌握 公开记录同类主题：[I07] [I13]

> **面试口述版**
>
> 种子控制随机源，不控制所有并行归约顺序、内核选择、低精度舍入或通信拓扑。要区分bitwise复现、容差数值一致与统计收敛一致；优化目标不同，验收标准不同。

背景：问题为什么会出现

浮点加法不满足结合律，分片、batch变化和kernel融合可改变归约树。MoE Top-k边界还会把微小误差变成不同专家选择；dropout、data shuffle与activation recompute也有各自RNG状态。

原理与推导：把关键条件说清楚

对纯等价算子优化，先测forward/backward与高精度参考误差；对融合/量化配方，再做固定batch短程训练和多seed任务评估。绝对误差、相对误差、cosine与异常值分位数应结合，不能只用一个全局平均掩盖小分量问题。

实现方案与验证

记录种子集合、数据顺序、软件/驱动/硬件、dtype、kernel选项、TF32和deterministic设置。相同环境重复，再逐项改变一个因素。严格确定性可能降低性能，应给出实际成本而不是承诺零代价。

常见追问与易错点

追问：结果不bitwise相同就错吗？答：不一定；先确定允许的语义与误差。追问：训练同batch与不同batch的结果必须一样吗？答：batchnorm、归一化目标、随机性与有限精度都可导致差异，需定义比较条件。

技术依据：[T23] [R21] [T05] | 口述、数值例子及实验建议为本册组织与推导。

05 数据、稳定性与恢复 / QUESTION 39

39 分布式 checkpoint 怎么保证一致？异步保存会不会读到更新中的参数？

P0 · 优先掌握 公开记录同类主题：[I06] [I16]

> **面试口述版**
>
> 完整续训快照至少含模型、optimizer、scheduler、RNG、训练步数和数据进度；分片保存要有全局metadata与完成标志。异步落盘必须基于一致快照，不能在参数继续更新时让后台直接读可变内存而得到混合版本。

背景：问题为什么会出现

分布式checkpoint的两个阶段是state staging与storage写入。前者决定拷贝一致性和显存/主存峰值，后者决定I/O成本。保存成功不是某一个rank文件存在，而是所有必要shard和manifest完整可读。

原理与推导：把关键条件说清楚

事务式思路：统一逻辑step边界→复制/冻结该版本状态→各rank写临时shard→校验与收齐→发布manifest。故障恢复只选择完整版本。改变world size需要重分片映射，参数名与optimizerstate对应关系必须可靠。

实现方案与验证

做故障注入：保存中杀rank、磁盘满、某shard缺失、版本不兼容，再验证不会从半成品恢复。恢复后与不间断训练比较短程loss/参数/数据样本ID，报告不能保证bitwise的条件。压缩、异步频率还要考虑恢复点目标和平均故障间隔。

常见追问与易错点

追问：只存model够不够？答：用于推理通常够，精确续训不够。追问：异步checkpoint免费吗？答：staging与I/O仍消耗资源，可能与H2D或网络竞争。

技术依据：[T22] [T23] | 口述、数值例子及实验建议为本册组织与推导。

06 性能测量与系统设计 / QUESTION 40

40 MFU、HFU、GPU utilization、tokens/s 分别说明什么?

P0 · 优先掌握 公开记录同类主题：[I06] [I13]

> **面试口述版**
>
> GPU utilization只表示采样窗口内设备忙碌程度，并不等于Tensor Core有效利用。MFU用有用模型FLOPs除理论峰值与时间；HFU口径可包含重计算等实际执行FLOPs。tokens/s还需说明有效非paddingtoken、总卡数和测量范围。

背景：问题为什么会出现

若重算变多，设备更忙且HFU可提高，但每秒完成的有效训练token可能下降。使用稀疏峰值当稠密分母、混用FP8与BF16峰值、只看最快rank，都会让数字失真。MFU公式需要精确的模型FLOPs口径。

原理与推导：把关键条件说清楚

教学例：同模型原step1s、优化后0.8s，global有效token固定，则吞吐提高25%、时间减少20%；二者不能都说“提高20%”。若优化同时增大padding，rawtokens/s上升却不一定是有效数据吞吐上升。

```
MFU = useful model FLOPs per step / (GPU_count × peak FLOP/s × step_seconds)
```

实现方案与验证

CUDA执行异步，CPU计时需正确同步或用CUDA events；分布式step按最慢rank和完整依赖边界衡量。报告warmup、计时次数、median/p95、shape分布和初始化是否计入。完整训练另报checkpoint、数据等待与故障重启占比。

常见追问与易错点

追问：MFU越高训练越划算吗？答：还需质量、总token与硬件成本；更快优化错误目标无意义。追问：只测100次kernel能代表训练吗？答：不能替代真实shape/通信/数据链路的端到端评估。

技术依据：[T37] [T25] [T26] | 口述、数值例子及实验建议为本册组织与推导。

06 性能测量与系统设计 / QUESTION 41

41 强扩展、弱扩展和 Amdahl 定律如何解释多卡不加速?

P1 · 进阶补充 公开记录同类主题：[I05] [I10]

> **面试口述版**
>
> 强扩展固定总工作量增加设备，弱扩展随设备增加工作量。强扩展时每卡GEMM变小且通信相对占比上升，效率常下降；Amdahl定律说明串行/不可扩展部分决定上限，但真实系统还会引入额外通信与容量效应。

背景：问题为什么会出现

若可并行部分占f，理想加速为1/[(1−f)+f/N]。例如f=0.9、N=8，理想约4.71倍，而非8倍。该模型假设工作量不变且没有新开销；多机训练需再叠加通信、同步和小batch效率变化。

原理与推导：把关键条件说清楚

有时超线性加速来自单卡原本需要offload或频繁重算，多卡容量让这些额外成本消失；不能因此声称算术本身违反Amdahl。固定globalbatch的强扩展和固定每卡batch的弱扩展也会改变优化器统计，比较需区分。

```
Speedup_ideal(N)=1 / [(1−f)+f/N]
```

实现方案与验证

画1/2/4/8…卡的step time、tokens/s、并行效率，分解F/B/comm/data。保持目标质量与数据预算，说明是否改变globalbatch及学习率。对于TP扩展，重点看GEMM维度和每层同步；DP扩展则关注梯度归约与数据供给。

常见追问与易错点

追问：卡数翻倍吞吐只涨30%就是NCCL问题吗？答：还可能是GEMM退化、数据瓶颈、PPbubble或CPU控制。追问：如何预测下一个规模？答：用当前分解模型与拓扑估计，再小规模验证，不直接线性外推。

技术依据：[T12] [T26] | 口述、数值例子及实验建议为本册组织与推导。

06 性能测量与系统设计 / QUESTION 42

42 手写题：一个不会错位、错误累积或漏同步的训练 step

P0 · 优先掌握 公开记录同类主题：[I12] [I16]

> **面试口述版**
>
> 训练step包括清梯度、前向、正确mask与shift的loss、反向累积、必要同步/unscale、裁剪、optimizer更新和scheduler推进。顺序和计数错误比不会调用API更常见；手写时先声明单卡/DDP及固定或变长batch假设。

背景：问题为什么会出现

附录给单卡因果CE和线性梯度校验，避免将不完整伪分布式代码当可直接投产脚本。对于固定等权microbatch，可平均各子loss；对全局tokenmean，分母必须覆盖整个累积窗口。

原理与推导：把关键条件说清楚

检查点：zero_grad不能放在每个累积microstep；optimizer.step只在窗口末；FP16裁剪在unscale后；scheduler步数对应实际optimizer更新；DDP最后一次同步不能被no_sync覆盖；全padding需显式处理而不是产生0/0。

实现方案与验证

测试“同一batch一次训练”与“拆成不等长度microbatch累积”更新前梯度一致。再测试冻结参数、unused branch、训练/eval切换及断点恢复。学习率为0应不改变参数，但优化器状态可能仍更新，测试预期必须具体。

常见追问与易错点

追问：训练loss和验证loss为何口径不同？答：dropout、mask、teacher forcing与数据分布可能不同，先统一统计。追问：手写CE能直接softmax再log吗？答：容易数值下溢，使用logsumexp/log_softmax。

技术依据：[T05] [T07] [T09] ｜ 口述、数值例子及实验建议为本册组织与推导。

06 性能测量与系统设计 / QUESTION 43

43 系统设计：7B 模型、8×80GiB 预算，该如何选 DDP / ZeRO / TP?

P0 · 优先掌握 公开记录同类主题：[I06] [I16]

> **面试口述版**
>
> 先做账本再选并行。这里人工假设每卡可用80GiB、单机高速互联、7B稠密全参训练，参数/梯度BF16加FP32master和Adam状态。仅模型状态未分片约104.3GiB/卡，普通DDP放不下；可从ZeRO-2/FSDP或结合TP评估。

背景：问题为什么会出现

8路ZeRO-2常驻粗估26.25GB≈24.45GiB；ZeRO-3粗估14GB≈13.04GiB。剩余空间不是全能给batch，还要激活、logits、collective、临时gather和碎片。长context可能让activation主导，需要checkpoint/CP，而不是只升级状态分片。

原理与推导：把关键条件说清楚

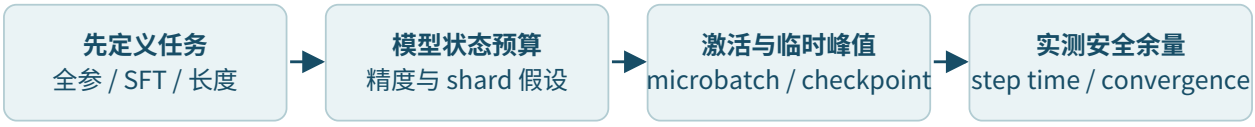
首版选择能满足容量且工程简单的方案：若Z2已装下目标microbatch并有良好overlap，额外Z3的参数gather可能不划算；若单层大或激活压力高，再比较TP与checkpoint。保留相同有效globaltoken与优化器配方，防止性能对比混入学习目标变化。

实现方案与验证

跑小批fixed-data正确性、单step峰值、几十step稳定性能、保存恢复和长程loss。列出每个候选的状态显存、峰值、step分解、有效tokens/s与失败边界。本题只有预算推导，没有声称在某型号GPU实测。

常见追问与易错点

追问：8卡总显存640GiB为何还OOM？答：不代表每个瞬时张量自动切8份，最大单元、复制与工作区约束仍在。追问：量化权重能直接解全参训练容量？答：需兼容训练配方，optimizer/master仍可能占主导。



算得下是必要条件，不是速度保证；卡内存按容量约定区分 GB 与 GiB。

示意图由本册绘制；展示机制，不代表实测性能。

技术依据：[T08] [T10] [T11] | 口述、数值例子及实验建议为本册组织与推导。

06 性能测量与系统设计 / QUESTION 44

44 系统设计：70B、128 卡、多节点训练如何规划与验收?

P1 · 进阶补充 公开记录同类主题：[I06] [I10] [I15]

> **面试口述版**
>
> 先问节点内卡数/互联、节点间网络、每卡可用显存、序列长度、globalbatch和交付时限，再给一个候选mesh，而不是背“70B就TP8PP4”。给出容量估算、通信主路径、pipelinebalance和恢复方案，并以实验淘汰不合适配置。

背景：问题为什么会出现

教学候选：假设16个8卡高速节点，TP=8尽量留在节点内，PP=4跨节点，DP=4，总128卡；CP=1。另一候选可降低TP增加DP或加入CP，取决于GEMM尺寸与长序列压力。没有具体网络/shape数据不能承诺哪个最快。

原理与推导：把关键条件说清楚

在均匀切层的粗略假设下，每rank承载P/(TP · PP)≈2.1875B参数；若此局部模型状态再按DP=4作完整分片，16P_local/4≈8.75GB，但这是不含gather、激活和非均匀层的理想常驻下界。必须回到逐stage账本，不能拿该数直接判断batch。

实现方案与验证

验收分四级：节点内/跨节点collective基准；小模型同mesh数值；目标模型短跑与故障注入；固定数据预算长跑与扩展效率。配置checkpoint频率、坏节点隔离、数据副本和断点重分片，记录平均恢复损失与资源成本。

常见追问与易错点

追问：CPU/GPU利用率很高就是规模化成功吗？答：还要MFU、time-to-quality、有效token和失败恢复成本。追问：MoE可照搬该mesh吗？答：EP和专家副本/热点改变通信与状态账本，必须重新规划。

技术依据：[T11] [T12] [T17] [T22] | 口述、数值例子及实验建议为本册组织与推导。

附录 A / CPU REFERENCE

A1. 因果 LM loss 与全局 token 归一化

各页函数共用 import torch 与 import torch.nn.functional as F。下列代码用于检查数学语义，不包含完整 RL trainer、分布式封装或生产级异常恢复。

```
def causal_lm_loss(logits: torch.Tensor,
                   labels: torch.Tensor) -> torch.Tensor:
    if logits.ndim != 3 or labels.shape != logits.shape[:2]:
        raise ValueError("expect logits[B,S,V], labels[B,S]")
    if logits.shape[1] < 2:
        raise ValueError("sequence too short")
    pred = logits[:, :-1].contiguous()
    target = labels[:, 1:].contiguous()
    if (target != -100).sum().item() == 0:
        raise ValueError("no supervised next-token labels")
    return F.cross_entropy(pred.reshape(-1, pred.shape[-1]),
                           target.reshape(-1), ignore_index=-100)

def token_normalized_local_loss(local_sum: torch.Tensor,
                                global_tokens: int,
                                dp_world: int) -> torch.Tensor:
    """Assumes subsequent DDP gradients are averaged over dp_world."""
    if global_tokens <= 0 or dp_world <= 0:
        raise ValueError("positive counts required")
    return dp_world * local_sum / global_tokens
```

> 先得到全局有效 token 数，再把本地 token loss 的和乘 DP world / global_count。代码假设后续 DDP 平均梯度；若后端只求和或采用别的缩放，系数必须相应修改。梯度累积跨 microbatch 时 global_count 应覆盖同一个 optimizer step。

附录 A / CPU REFERENCE

A2. Attention 分区的 O / LSE 合并

每个分区计算自己的归一化 attention 输出及 logsumexp。所有分区采用同一个 query、同一缩放与一致 mask 语义；不同分区的 KV 范围不应重复计数。

```
def merge_attention(o: torch.Tensor,
                    lse: torch.Tensor) -> torch.Tensor:
    """o[r,...,d] is normalized per-partition output; lse[r,...].
    Empty/all-masked partitions use lse=-inf and finite zero output.
    """
    if o.shape[:-1] != lse.shape:
        raise ValueError("o/lse partition shapes do not match")
    total = torch.logsumexp(lse, dim=0)
    safe_total = torch.where(torch.isfinite(total), total,
                             torch.zeros_like(total))
    weight = torch.where(torch.isfinite(lse),
                         torch.exp(lse - safe_total),
                         torch.zeros_like(lse))
    return (o * weight.unsqueeze(-1)).sum(dim=0)
```

> 空分区规定 lse=-inf、O=0，避免 0×NaN 污染输出。这里的 CPU 检查验证数学归并，不代表跨 rank 通信已经实现。有限精度和不同归约树会改变舍入顺序。

附录 A / TRAINING LOOP

A3. 从局部数学走到训练循环

下面是组织顺序的伪代码，不是可直接启动的完整训练程序。抽象函数需要由具体框架提供，并且应适配 DDP、FSDP 或 Megatron 的真实归约语义。

```
# Illustrative update structure, not executed as a GPU/DDP test.
optimizer.zero_grad(set_to_none=True)
# Count valid supervised tokens across this update and DP group.
global_tokens = count_and_allreduce_tokens(microbatches)
for j, batch in enumerate(microbatches):
    last = (j == len(microbatches) - 1)
    # no_sync must cover BOTH forward and backward for DDP.
    with sync_context(model, synchronize=last):
        with torch.autocast("cuda", dtype=torch.bfloat16):
            logits = model(batch["input_ids"])
            loss_sum = shifted_masked_ce_sum(logits, batch["labels"])
            loss = dp_world * loss_sum / global_tokens
        loss.backward()
# All ranks must reach compatible collectives and update decisions.
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm)
optimizer.step()
scheduler.step()
# FSDP/TP may require sharding-aware global norm computation.
# FP16+GradScaler: scale(loss); unscale BEFORE clipping;
# advance scheduler according to actual successful updates.
```

手写题的验收标准

先用无 dropout 的小模型，对照单批次与多 microbatch 的梯度；再覆盖不同 rank 的有效 token 数不等。检查 unscale 与 clipping 顺序、no_sync 范围、是否漏掉最后一组同步，以及跳步时 scheduler 的推进。

分片参数下，普通局部 clip_grad_norm_ 不一定计算到所需的全局范数；不应把这段单副本写法不加适配搬入所有并行框架。代码中的“global”计数也必须限定到同一个数据副本组，而非错误地将 TP ranks 重复计入。

> 这页未在 GPU/DDP 上执行。已执行的检查是下一页列出的 CPU 公式与梯度对照；测试范围与未验证范围刻意分开。

附录 B / ACTUAL CHECKS

实际 CPU 验证结果

执行环境：CPU，PyTorch 2.10.0+cpu，float64。检查日期 2026-10-31。本册相关检查共 5 组、78 个测试情形；“情形”不是独立实验或性能样本。

检查组 / 情形数	最大绝对误差	验证内容与限制
GEMM backward 与 TP 分解 20 情形 · 通过	1.776e-15	dX/dW 对 autograd；column concat 与 row partial-sum 对稠密前向。
Causal CE 的 shift/mask 2 情形 · 通过	0.000e+00	手工逐 token logprob 对交叉熵；最后一个输入位置不预测序列外 label。
变长 token 的 DDP 梯度归一化 20 情形 · 通过	1.776e-15	CPU 模拟 3 rank 的梯度平均；局部均值误法出现最大差异 3.416641，不是多进程执行。
分块 Attention 的 O/LSE 合并 31 情形 · 通过	2.398e-14	不等长分块、极端分数、全 mask；与完整 softmax attention 一致。
显存账本与理想 PP bubble 5 情形 · 通过	0.000e+00	7B、DP8、16 B/param 假设：112/38.5/26.25/14 GB；p4,m8 bubble=3/11。

这些结果证明什么？

在所列输入、dtype 与边界条件下，代码输出和手工数学参考一致；相关梯度方向通过 autograd 对照。它们能发现符号、shift、归一化与边界错误，但不能证明完整系统在任意输入下正确。

没有验证的内容

没有进行 CUDA kernel 数值回归、多 GPU collective 执行、训练收敛比较、吞吐或成本测量。CPU 模拟 rank 的梯度平均，不等于实际启动了多进程 DDP。所有硬件选型与调参建议仍需在目标系统上运行。

> 复现时，先在小规模 float64 / FP32 reference 上检查语义，再进入 BF16/FP8 和分布式路径。误差阈值要结合数值尺度与下游质量，不要拿这页 float64 误差阈值直接约束低精度大模型。

附录 C / PRACTICE PATH

怎么用这本书准备面试

三条连续追问链

主线	建议题号	应当交付的能力
数学与账本	01 → 04 → 05 → 08 → 10 → 14	从 loss 和梯度出发，算清参数/梯度/优化状态。
并行与通信	13 → 17 → 19 → 21 → 23 → 26	画出 rank 组与矩阵切分，说明关键路径。
诊断与系统	35 → 36 → 37 → 39 → 40 → 43 → 44	拿到 trace / 显存数据后，提出可验证的系统方案。

一场 45 分钟的自测

前 5 分钟讲一个真实项目：问题、约束、基线、你的改动与验证。接下来 15 分钟选一条追问链，不看答案口述并推导；再用 15 分钟完成一道代码或预算题；最后 10 分钟让对方改变一个假设，看你是否能调整方案。

给答案打分，而不是给自己贴标签

层次	检查方法
1 · 定义	能否不用框架名字堆砌，准确说明输入、输出和目标。
2 · 推导	能否写出关键公式，说明归一化、边界和假设。
3 · 实现	能否定位 mask、张量布局、通信组或状态生命周期。
4 · 验证	能否设计对照、监控失败信号，解释哪些事实尚未验证。

项目表达模板

“原问题在什么条件下出现 → 我如何测量 → 排除了哪些原因 → 做了什么改动 → 正确性/质量如何验证 → 端到端收益与代价是什么”。没有实测的数据明确称作估算，不把本册教学配置包装成自己的项目经历。

> 复习的终点不是逐字背完：应当能在 batch 变大、序列变长、并行组变化、奖励失效或一个 rank 掉线时，重新推导系统行为。

来源档案 / INTERVIEW RECORDS

公开面经与题库

公司名为原作者或转载者的归属描述。A/B 不是面试难度评级；原帖里的答案、招聘推断和营销内容未自动采纳。

[I02] 百度｜大模型后训练一面

A：作者自述；正文写面试日 2026-03-18。

公开主题：PPO clip、重要性采样、GAE、GRPO 归一化、分布式显存。

关联题号：08、14（同类主题映射，非逐字题面）。

https://www.nowcoder.com/discuss/863890669662674944

[I05] 小米汽车｜深度学习训练框架优化

A：作者自述；页面日期 2024-11-23。

公开主题：TP 行列切分、SP、FlashAttention、CUDA 存储、AllReduce。

关联题号：02、04、07、09、13、17、18、21、23、24、25、29、30、41（同类主题映射，非逐字题面）。

https://www.nowcoder.com/discuss/689825041512419328

[I06] 科大讯飞｜框架与平台方向

A：作者自述；页面日期 2024-11-23。与 I05 同作者，不作独立作者计数。

公开主题：Adam/LAMB、精度、ZeRO、TP/PP、通信重叠与集群容错。

关联题号：05、06、08、09、10、12、13、14、15、16、19、20、23、24、25、28、35、36、37、39、40、43、44（同类主题映射，非逐字题面）。

https://www.nowcoder.com/feed/main/detail/7e188fe225e34173bde7f1751866abae

---

[I07] 匿名多场｜AI Infra 复盘

A：多场自述；公司匿名，显示 03-03，不推定年份。

公开主题：LoRA、CUDA、TMA、FlashAttention、torchrun 与项目追问。

关联题号：03、04、11、29、30、31、32、34、38（同类主题映射，非逐字题面）。

https://www.nowcoder.com/feed/main/detail/d695614a06424c148c04586ac3a66e78

[I10] 腾讯混元｜AI Infra 一面转载

B：二次整理；站点标注 2026-10，未独立认证原始面试。

公开主题：TP 通信、GQA、DP/TP、PP bubble、CP/SP。

关联题号：02、07、17、18、19、20、21、22、26、41、44（同类主题映射，非逐字题面）。

https://mj.mianlingai.com/interview/tencent-hunyuan-ai-infra-2922039/

[I11] 字节大模型岗｜跨平台汇总

B：CSDN 汇编，宣称汇集多平台；不按每个平台重复计次。

公开主题：SFT、PPO/DPO/GRPO、数据、训练与推理显存；仅供主题发现。

关联题号：01、02、06、33（同类主题映射，非逐字题面）。

https://blog.csdn.net/qq_45717425/article/details/160315245

来源档案 / INTERVIEW RECORDS

公开面经与题库 · 续

[I12] MiniMax｜AI Infra 题目转载

B：AIInfraGuide 转载；2026-04-17 为站点日期，不当作面试日。

公开主题：SFT loss 手写、LoRA、PPO/DPO/GRPO 与工具调用。

关联题号：01、03、27、33、34、42（同类主题映射，非逐字题面）。

https://caomaolufei.github.io/AIInfraGuide/interview/minimax-ai-infra-一面/

[I13] LLM RL Infra Interview｜开源题库

B：社区题库，非独立面试记录。

公开主题：训练/RL Infra 知识范围检查；不继承其“高频”统计。

关联题号：07、12、15、16、22、26、28、31、32、35、36、37、38、40（同类主题映射，非逐字题面）。

https://github.com/XFWang522/llm-rl-infra-interview

[I15] AI Infra 面试收录｜博客园

B：混合岗位指南和模拟题；不作为某家公司实际出题证据。

公开主题：系统设计主题补充；薪资、招聘比例、公司题频未采用。

关联题号：44（同类主题映射，非逐字题面）。

https://www.cnblogs.com/xmwblogs/p/19669357

[I16] 网易互娱｜面经与内推帖

B：转述/内推；发布日与文内月日关系不清，不推定面试年。

公开主题：Megatron/DeepSpeed、ZeRO、SFT mask、显存与中断恢复。

关联题号：01、08、14、27、33、39、42、43（同类主题映射，非逐字题面）。

https://www.nowcoder.com/discuss/781170128720289792

[I17] 蚂蚁｜AI Infra 三面转载

B：AIInfraGuide 转载；非公司官方题。

公开主题：FP8、KV cache、量化与项目精度/性能验证。

关联题号：10、11（同类主题映射，非逐字题面）。

https://caomaolufei.github.io/AIInfraGuide/interview/蚂蚁-ai-infra-实习-三面/

原始依据 / PRIMARY REFERENCES

论文与官方技术依据

算法以论文与作者项目为依据；实现细节以链接所示官方版本为准。网页访问日 2026-10-31。代码附录实际运行版本另见验证页，不与最新在线文档版本混为一谈。

[R21] Megatron Core：MoE Router Replay 设计
https://docs.nvidia.com/megatron-core/developer-guide/latest/api-guide/router_replay.html

[T01] Attention Is All You Need
https://arxiv.org/abs/1706.03762

[T02] Root Mean Square Layer Normalization
https://arxiv.org/abs/1910.07467

[T03] GLU Variants Improve Transformer
https://arxiv.org/abs/2002.05202

[T04] RoFormer / RoPE
https://arxiv.org/abs/2104.09864

[T05] PyTorch：Autograd mechanics
https://docs.pytorch.org/docs/2.15/notes/autograd.html

[T06] PyTorch：Optimizers
https://docs.pytorch.org/docs/2.15/optim.html

[T07] PyTorch：AMP examples
https://docs.pytorch.org/docs/2.15/notes/amp_examples.html

[T08] DeepSpeed：ZeRO 分阶段教程
https://www.deepspeed.ai/tutorials/zero/

[T09] PyTorch：DDP 设计
https://docs.pytorch.org/docs/2.15/notes/ddp.html

[T10] PyTorch：FSDP2 教程
https://docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html

[T11] Megatron Core：Parallelism Strategies
https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/parallelism-guide.html

[T12] Megatron-LM：TP/PP/DP 大规模训练论文
https://arxiv.org/abs/2104.04473

[T13] Megatron Core：Context Parallel
https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/features/context_parallel.html

原始依据 / PRIMARY REFERENCES

论文与官方技术依据 · 续

[T14] DeepSpeed Ulysses：原项目说明
https://github.com/deepspeedai/DeepSpeed/blob/master/blogs/deepspeed-ulysses/README.md

[T15] Ring Attention
https://arxiv.org/html/2310.01889

[T16] NCCL：Collective Operations
https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

[T17] NCCL：Troubleshooting
https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/troubleshooting.html

[T18] FlashAttention：IO-aware exact attention
https://arxiv.org/html/2205.14135v2

[T19] Transformer Engine：FP8 / MXFP8 primer
https://nvidia.github.io/TransformerEngine/examples/fp8_primer.html

[T20] PyTorch：Activation checkpoint
https://docs.pytorch.org/docs/2.15/checkpoint.html

[T21] PyTorch：CUDA semantics、memory、graphs
https://docs.pytorch.org/docs/2.15/notes/cuda.html

[T22] PyTorch：Distributed Checkpoint
https://docs.pytorch.org/docs/2.15/distributed.checkpoint.html

[T23] PyTorch：Reproducibility
https://docs.pytorch.org/docs/2.15/notes/randomness.html

[T24] PyTorch：DataLoader 与采样器
https://docs.pytorch.org/docs/2.15/data.html

[T25] PyTorch：Profiler 教程
https://docs.pytorch.org/tutorials/recipes/recipes/profiler_recipe.html

[T26] CUDA C++ Best Practices Guide
https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html

[T27] LoRA：Low-Rank Adaptation
https://arxiv.org/html/2106.09685

原始依据 / PRIMARY REFERENCES

论文与官方技术依据 · 续

[T28] QLoRA：Quantized LLM Finetuning
https://arxiv.org/html/2305.14314

[T29] DeepGEMM：Grouped GEMM 接口与布局
https://github.com/deepseek-ai/DeepGEMM

[T30] Hopper Tuning Guide
https://docs.nvidia.com/cuda/hopper-tuning-guide/index.html

[T31] Blackwell Tuning Guide
https://docs.nvidia.com/cuda/blackwell-tuning-guide/index.html

[T32] ZeRO 原论文
https://arxiv.org/abs/1910.02054

[T33] PyTorch FSDP 系统论文
https://arxiv.org/abs/2304.11277

[T34] GPipe：流水线训练
https://arxiv.org/abs/1811.06965

[T35] AdamW：Decoupled Weight Decay
https://arxiv.org/abs/1711.05101

[T36] LAMB：Large Batch Optimization
https://arxiv.org/abs/1904.00962

[T37] Scaling Laws for Neural Language Models
https://arxiv.org/abs/2001.08361

[T38] Zero Bubble Pipeline Parallelism
https://arxiv.org/abs/2401.10241

[T39] PyTorch：Introduction to torch.compile
https://docs.pytorch.org/tutorials/intermediate/torch_compile_tutorial.html